



Algorithms and Data Structures

Sujeong Kim



UNIVERSITY OF
MARYLAND

DFS

Topological Sort

Connected Components

Slide Credit: These slides are adapted from materials by Prof. Manocha with modifications and additions by Sujeong Kim.

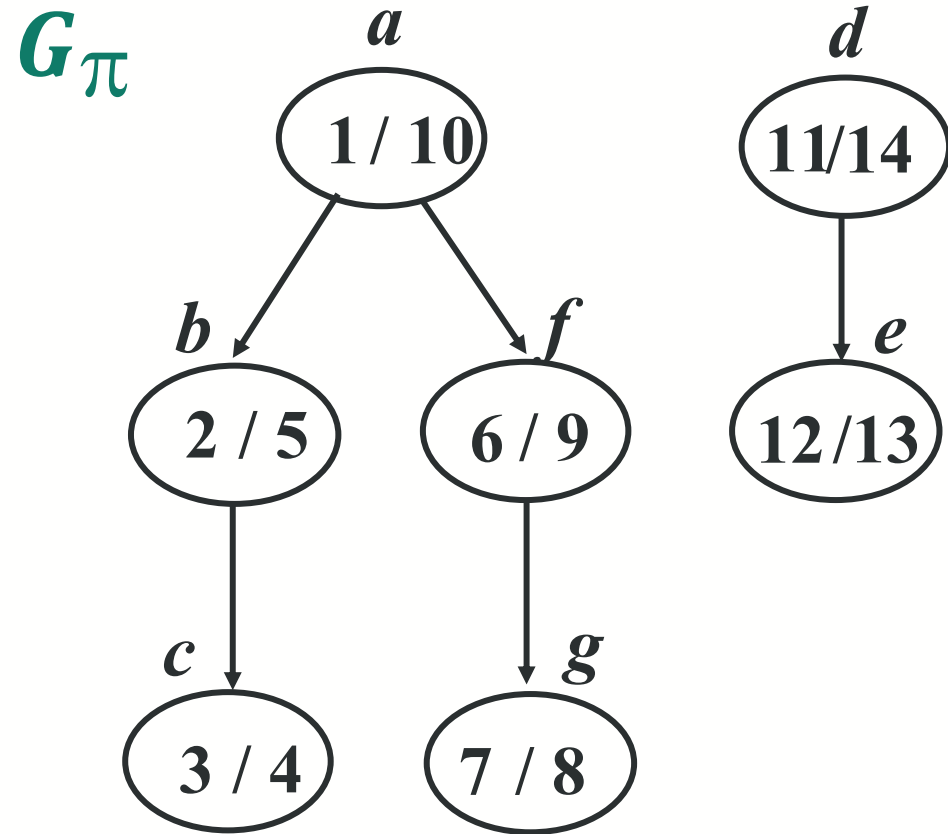
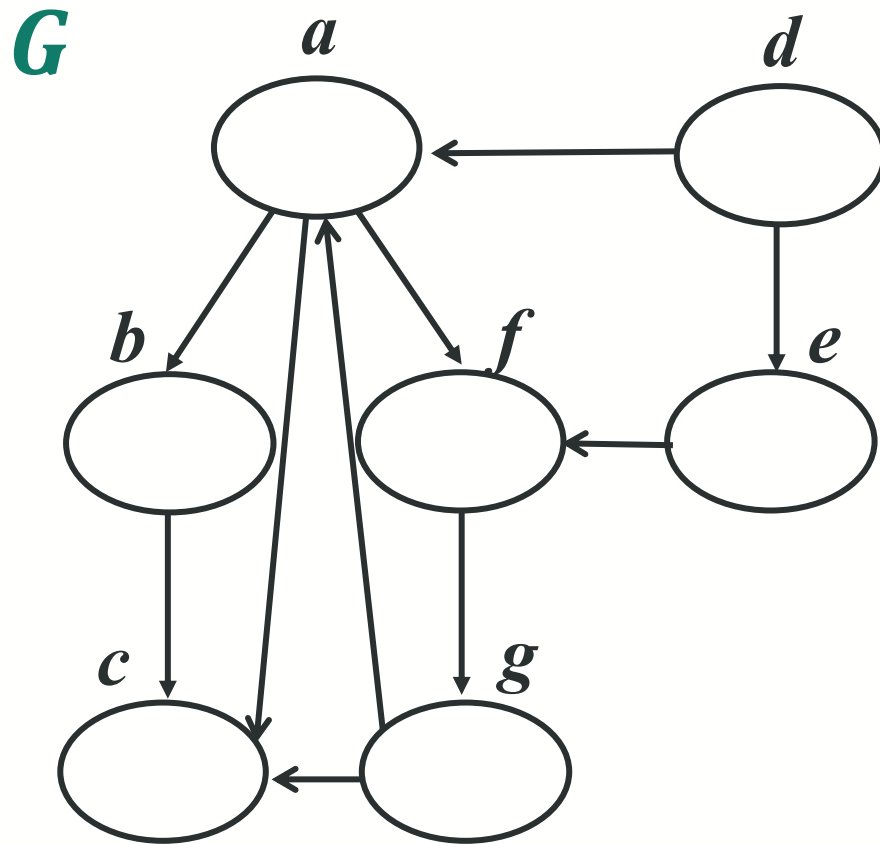
Properties of DFS

- Predecessor subgraph G_π forms a forest of trees (the structure of a depth-first tree mirrors the structure of DFS-Visit)
- The discovery and finishing time have **parenthesis structure**, i.e. the parentheses are properly nested. (See the figures and next theorem)

Valid Parentheses Structure

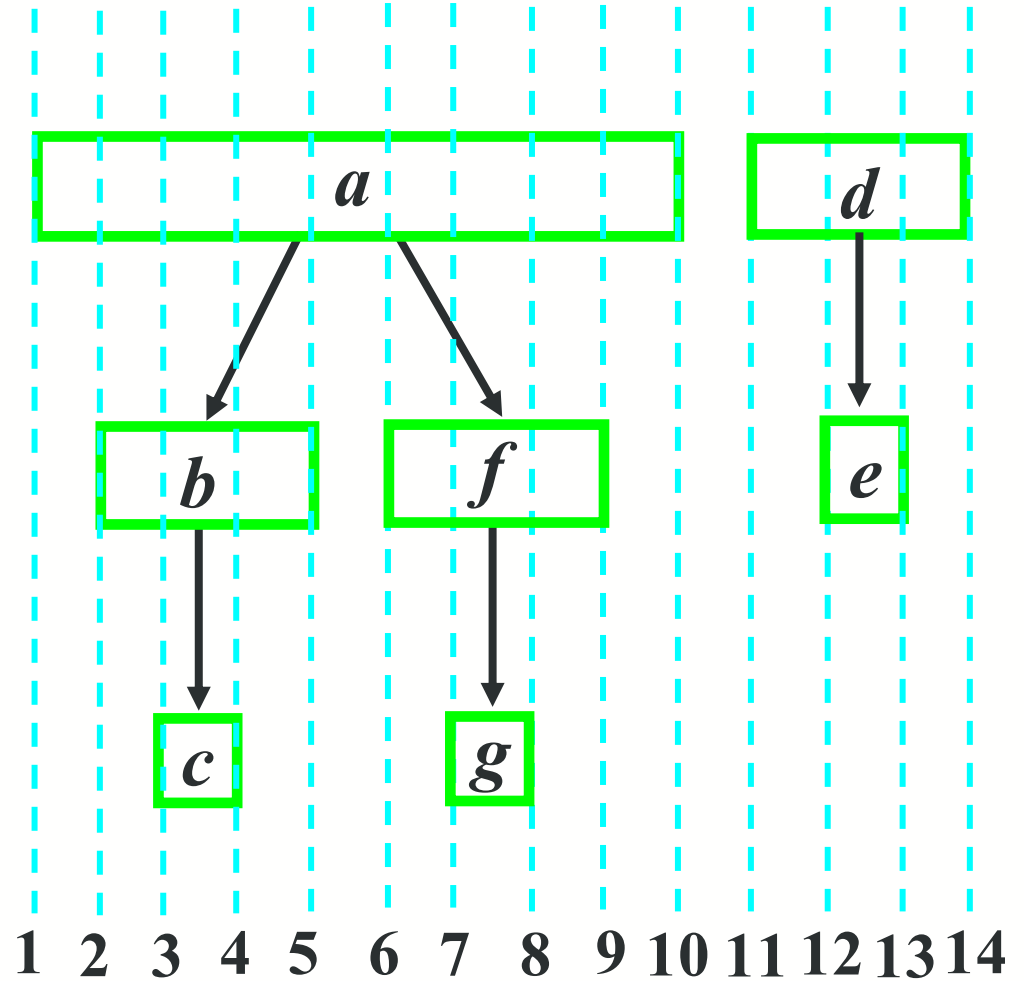
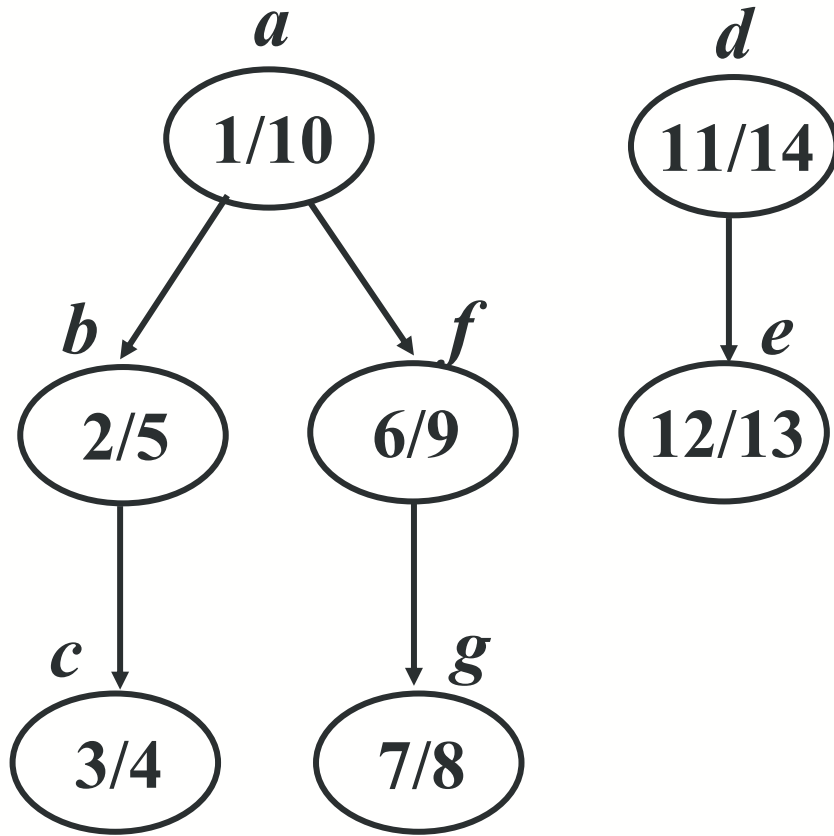
- a) `()`
- b) `[]`
- c) `([])`
- d) `([])`
- e) `({})[]`
- f) `{()}()`
- g) `{()}[]`
- h) `()[]{}`

DFS & Parenthesis Lemma



$(a(b(c\ c)b)(f(g\ g)f)a)(d(e\ e)d)$

DFS & Parenthesis Lemma



Parenthesis Theorem

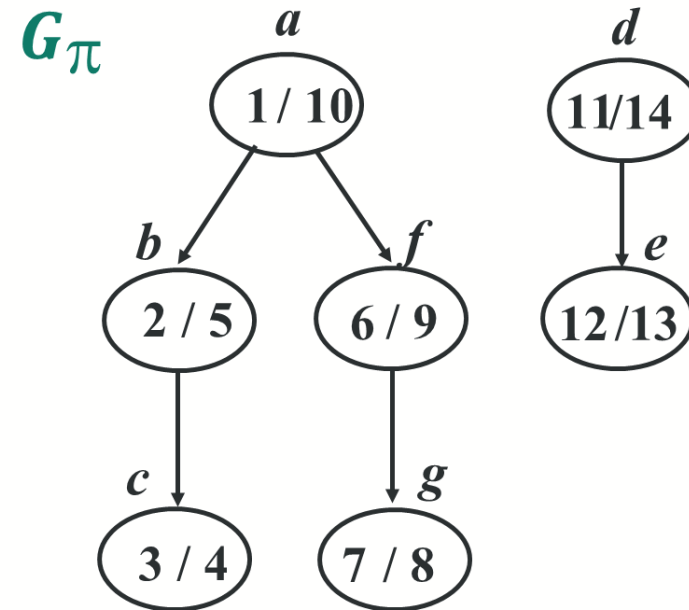
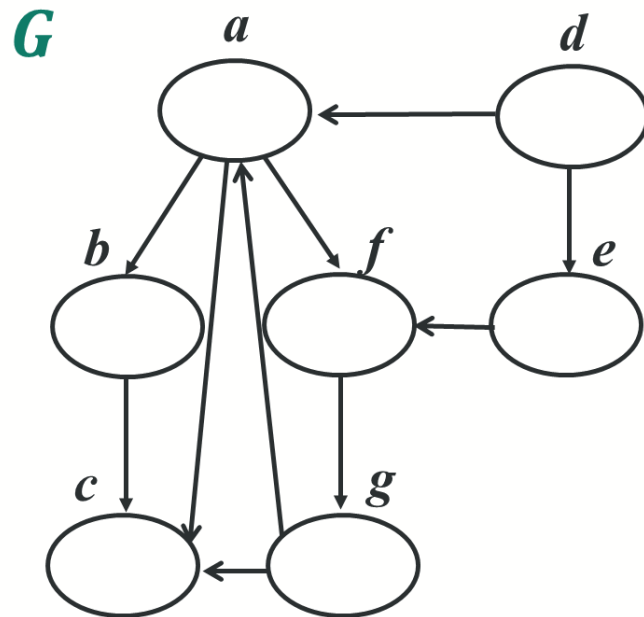
- In DFS, the timestamps $d[u]$ (discovery time) and $f[u]$ (finishing time) for a vertex u encode critical structural information about the graph.
- In any DFS of a graph $G = (V, E)$, for any two vertices u and v , exactly one of the following holds:
 - the interval $[d[u], f[u]]$ and $[d[v], f[v]]$ are entirely disjoint
 - the interval $[d[u], f[u]]$ is contained entirely within the interval $[d[v], f[v]]$, and u is a descendant of v in the depth-first tree, or
 - the interval $[d[v], f[v]]$ is contained entirely within the interval $[d[u], f[u]]$, and v is a descendant of u in the depth-first tree

Nesting of Descendant' Intervals

- Vertex v is a proper descendant of vertex u in the depth-first forest for a (direct or undirected) graph G if and only if $d[u] < d[v] < f[v] < f[u]$
 - u is discovered before $v \rightarrow d[u] < d[v]$
 - v finishes before $u \rightarrow f[v] < f[u]$
- Proof follows directly from Parenthesis Theorem (see the complete proof in the textbook CLRS 4th ed., Chapter 20.3)

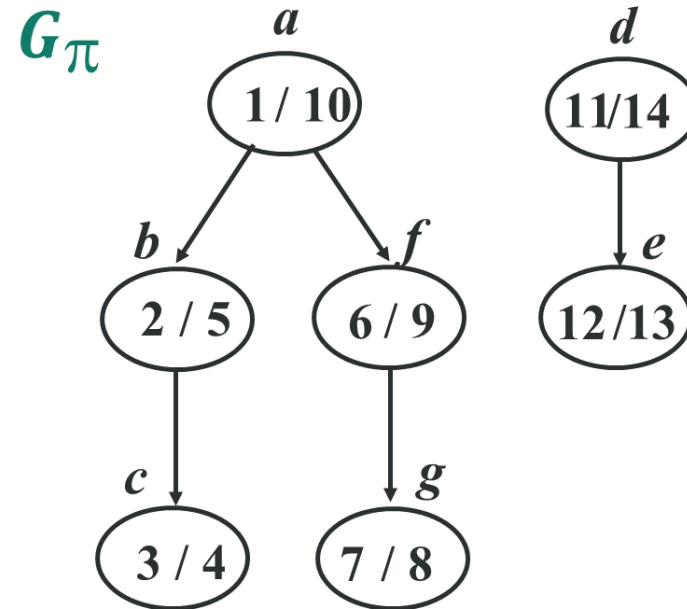
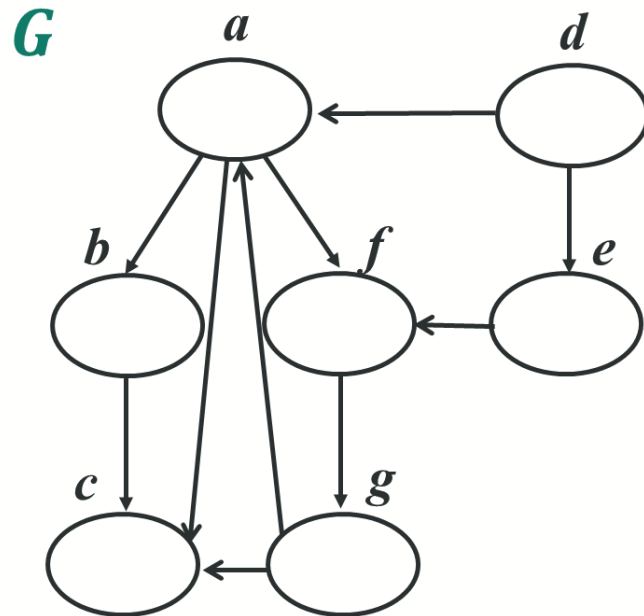
White-Path Theorem

- In a depth-first forest of a graph G , vertex v is a descendant of vertex u if and only if at the time $d[u]$ that the search discovers u , vertex v can be reached from u along a path consisting entirely of white vertices.



White-Path Theorem

- When DFS discovers a node u in G , it will explore every unvisited vertex reachable from u



White-Path Theorem

- In a depth-first forest of a graph G , vertex v is a descendant of vertex u if and only if at the time $d[u]$ that the search discovers u , vertex v can be reached from u along a path consisting entirely of white vertices.

Proof:

⇒ Assume v is a descendant of u and use Corollary on Nesting of Descendant's Interval

- If v is a descendant of u , then DFS discovered v during the recursive exploration of u .
- By the Corollary on Nesting of Descendant's interval: $d[u] < d[v] < f[v] < f[u]$
 - when DFS discovered u , v had not been discovered yet
 - so v was white at time $d[u]$
 - every vertex on the path from u to v was also undiscovered (white), because DFS discovers them in order as it walks the path

White-Path Theorem

- In a depth-first forest of a graph G , vertex v is a descendant of vertex u if and only if at the time $d[u]$ that the search discovers u , vertex v can be reached from u along a path consisting entirely of white vertices.

Proof:

⇐ By contradiction: suppose that v is reachable from u but v doesn't become a descendant of u . Use Parenthesis Theorem and NDI Corollary.

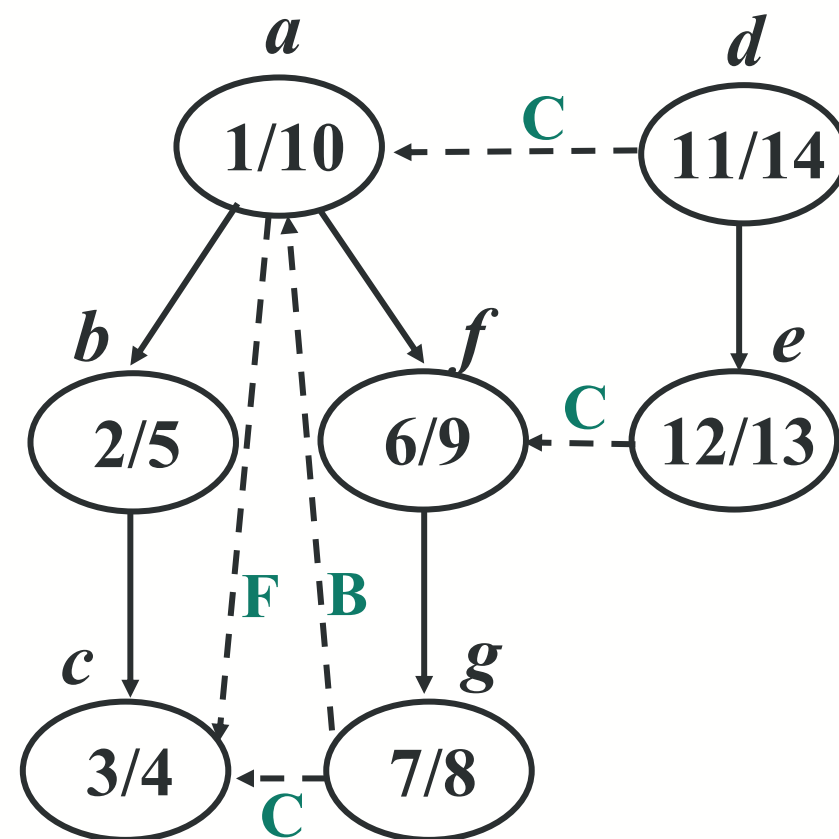
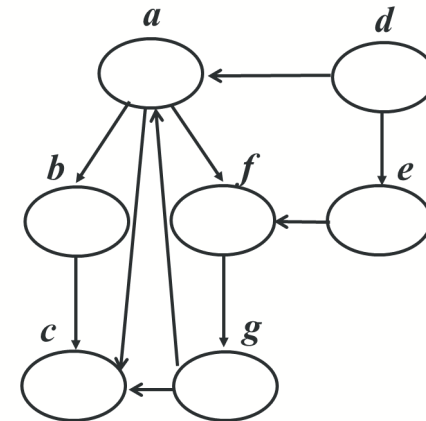
- At time $d[u]$, v is reachable from u by a path of white vertices
- But v does not become a descendant of u
 - case 1: v is discovered before u , contradicting DFS order.
 - case 2: v is discovered in a different DFS tree, that means:
 - DFS finished exploring all reachable vertices from u
 - Then later started a new DFS tree from some other root and discovered v there

Classification of Edges

DFS can be used to classify edges of G :

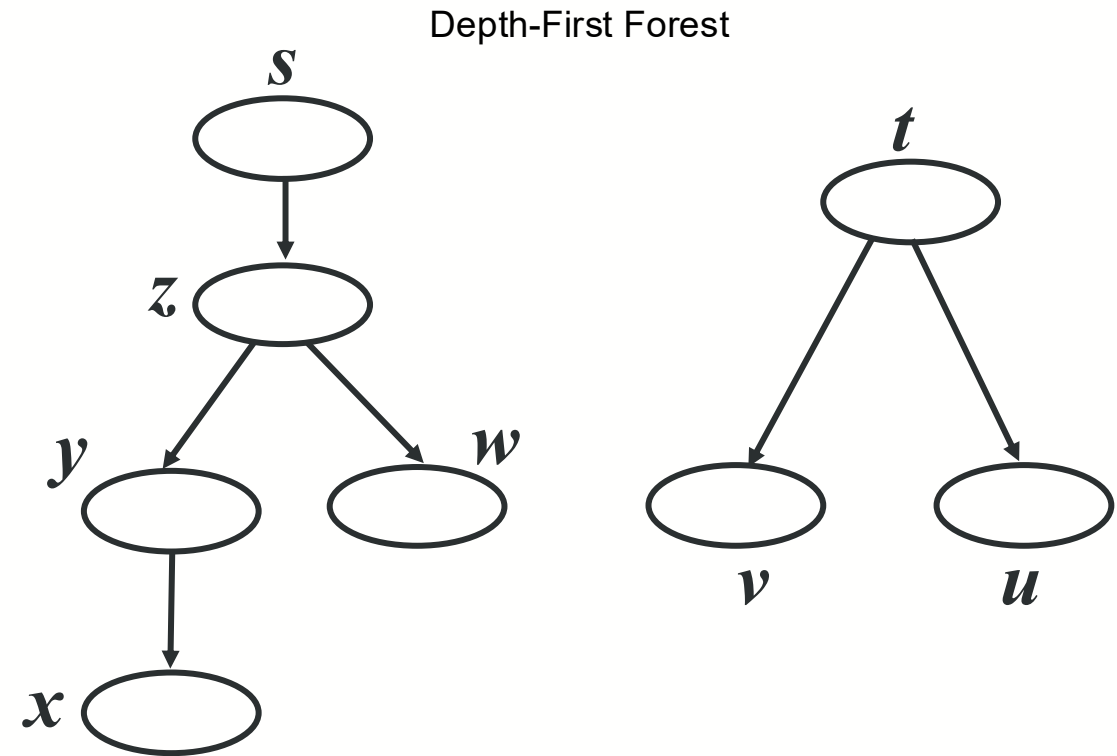
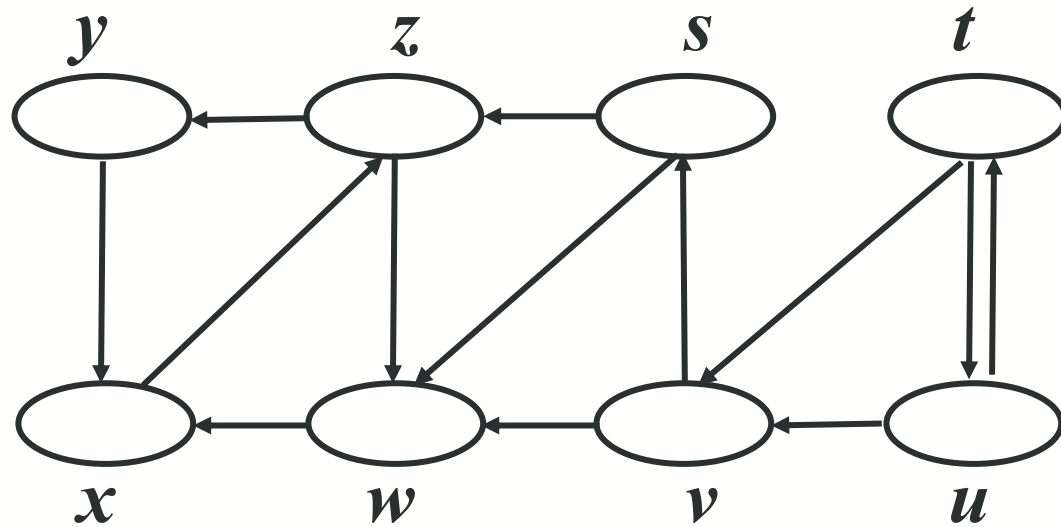
- 1. Tree edges:** edges in the depth-first forest G_π . Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) .
- 2. Back edges:** edges (u, v) connecting a vertex u to an ancestor v in a depth-first tree. Self-loops are considered to be back edges.
- 3. Forward edges:** nontree edges (u, v) connecting a vertex u to a descendant v in a depth-first tree.
- 4. Cross edges:** all other edges.

These ancestor-descendant relationships are based on the traversal path taken during the DFS, not the original graph G .

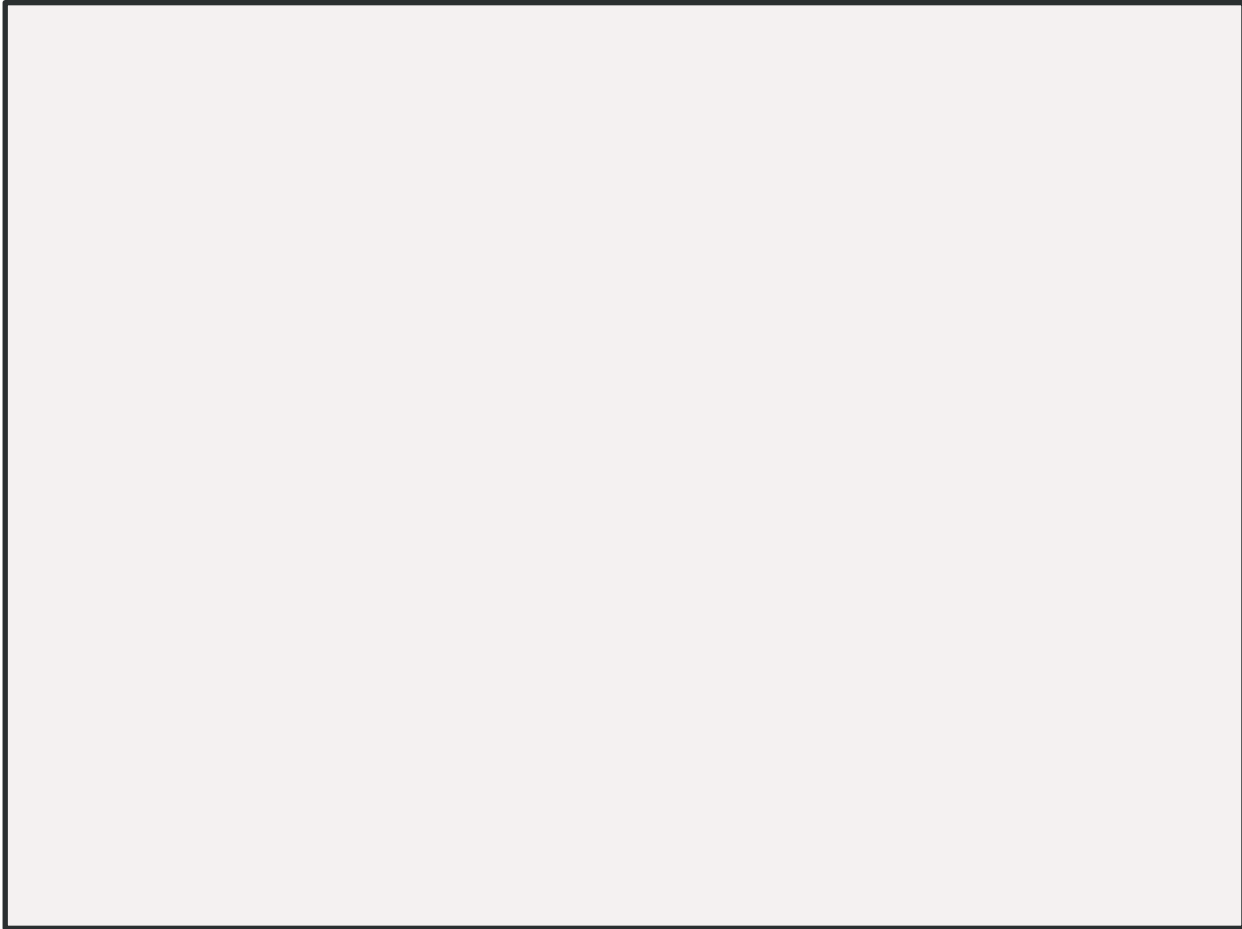


(Try it yourself) Classification of Edges

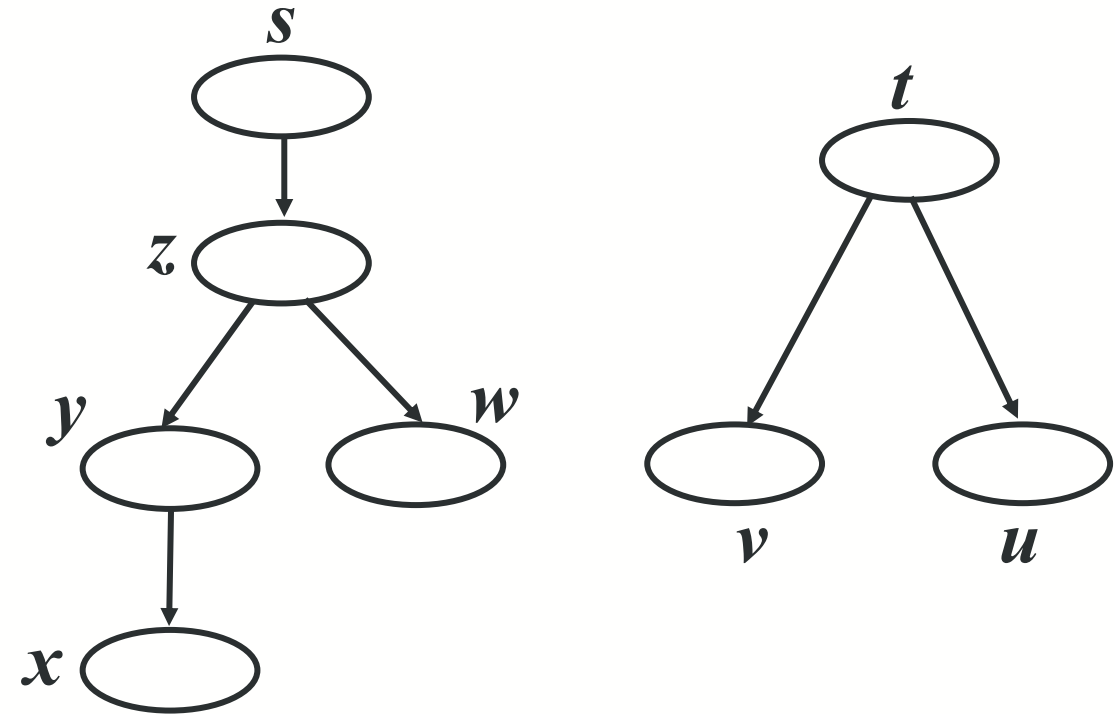
- Using the depth-first forest (DFS trees) provided, classify each edge of the original graph.



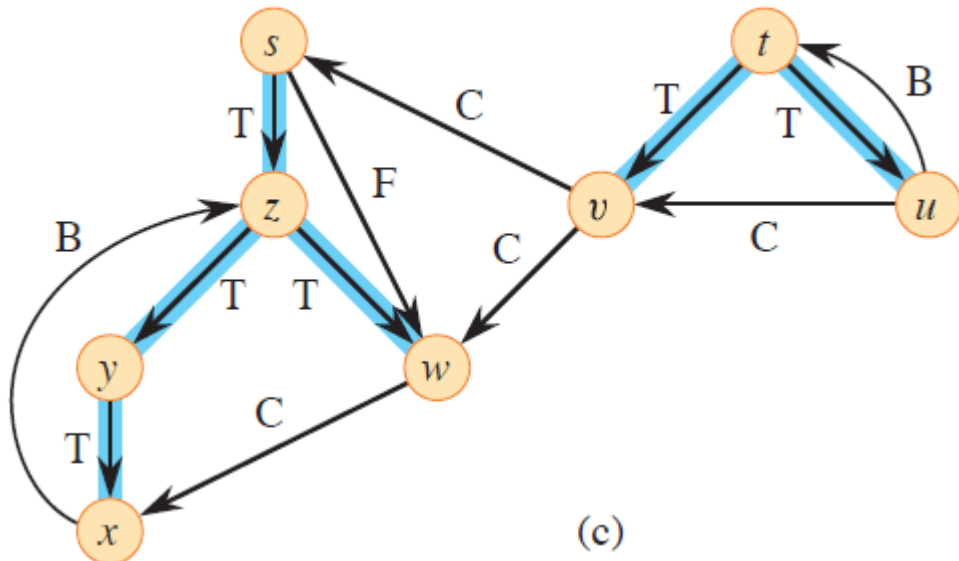
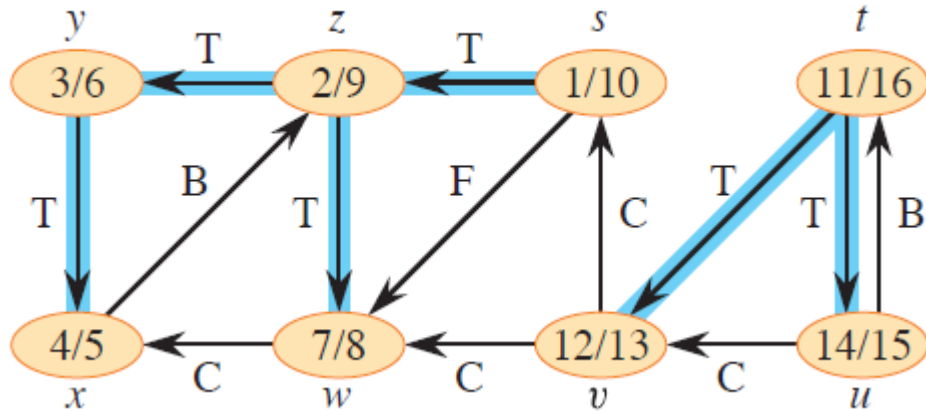
Classification of Edges



Depth-First Forest



Classification of Edges



DFS-Visit(u)

$color[u] \leftarrow \text{GRAY}$

∇ White vertex u has been discovered

$d[u] \leftarrow ++time$

for each vertex $v \in Adj[u]$

do if $color[v] = \text{WHITE}$

then $\pi[v] \leftarrow u$

Print("Tree Edge", (u, v))

DFS-Visit(v)

else if $color[v] = \text{GRAY}$

then print("Back Edge:", (u, v))

else if $color[v] = \text{BLACK}$

then if $d[u] < d[v]$

then print("Forward Edge:", (u, v))

then print ("Cross Edge:", (u, v))

$color[u] \leftarrow \text{BLACK}$

∇ Blacken u ; it is finished.

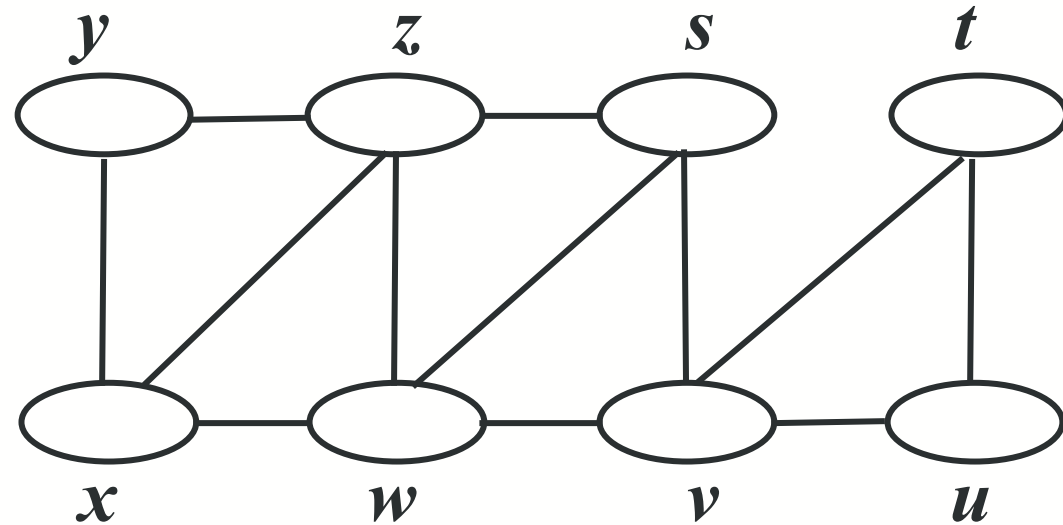
$f[u] \leftarrow time++$

Theorem

- In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.

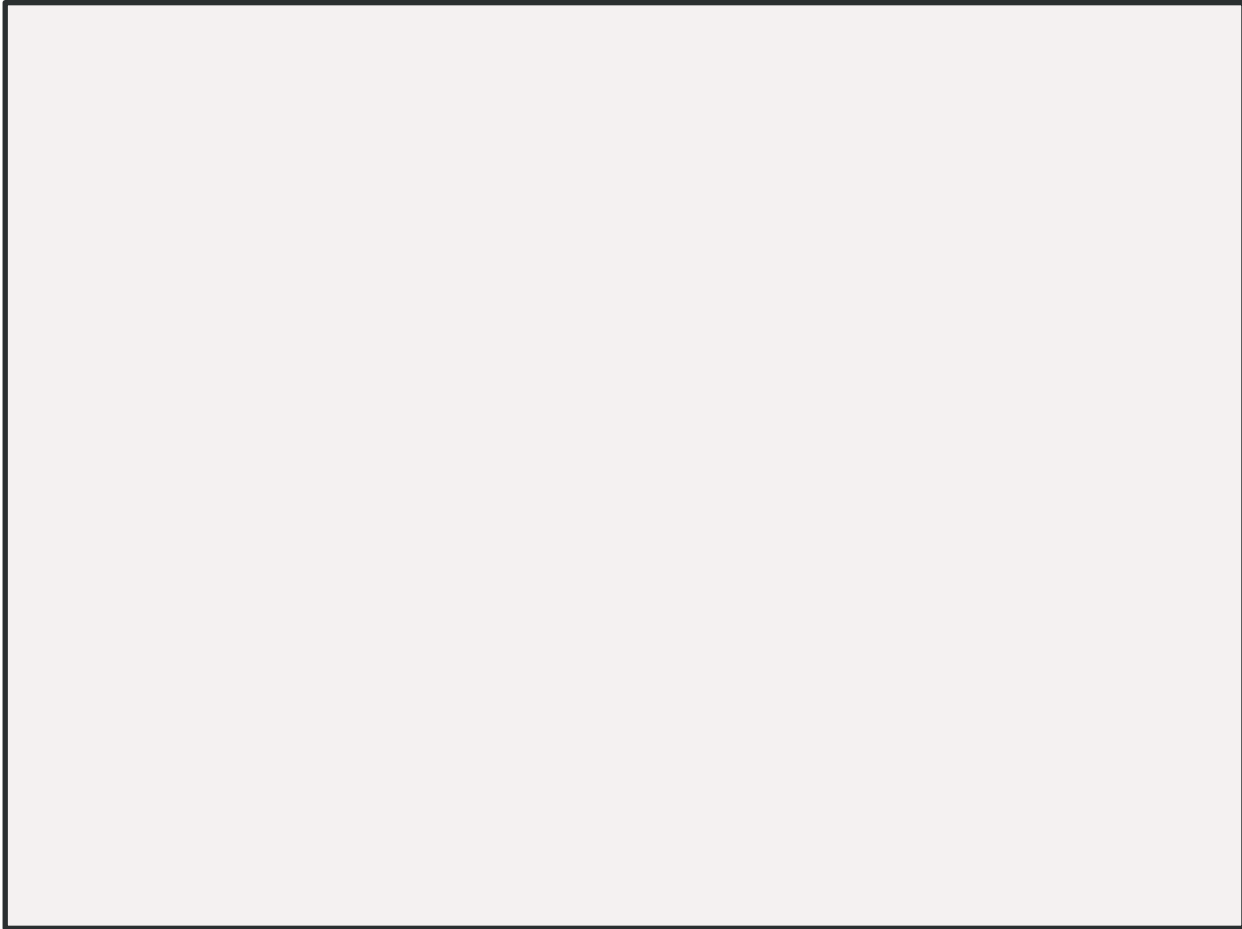
(Try it yourself) Depth-First Tree of an Undirected Graph

- Perform DFS on the graph and draw the DFS trees

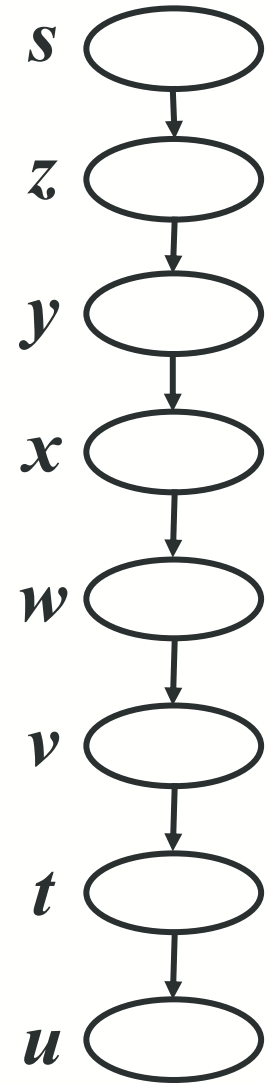


Depth-First Tree

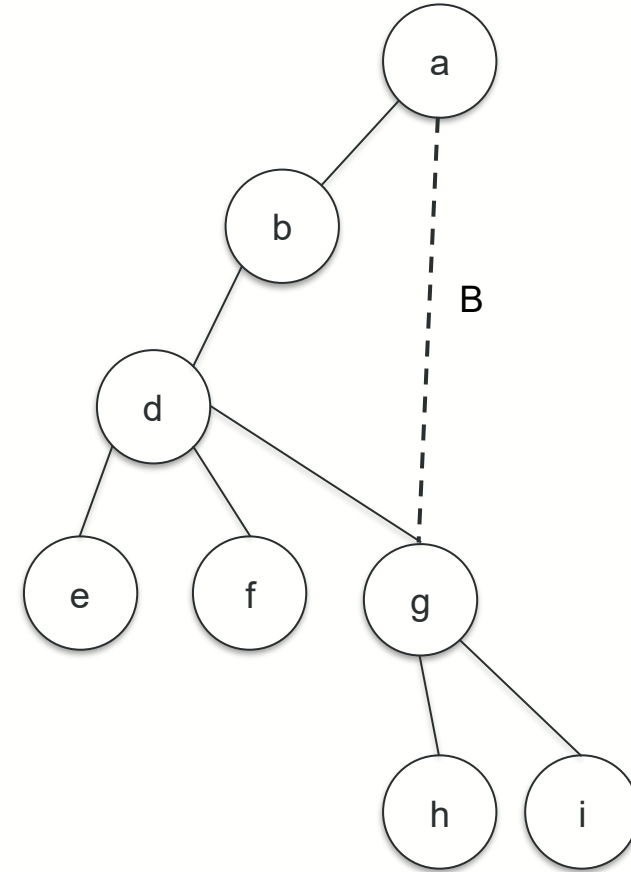
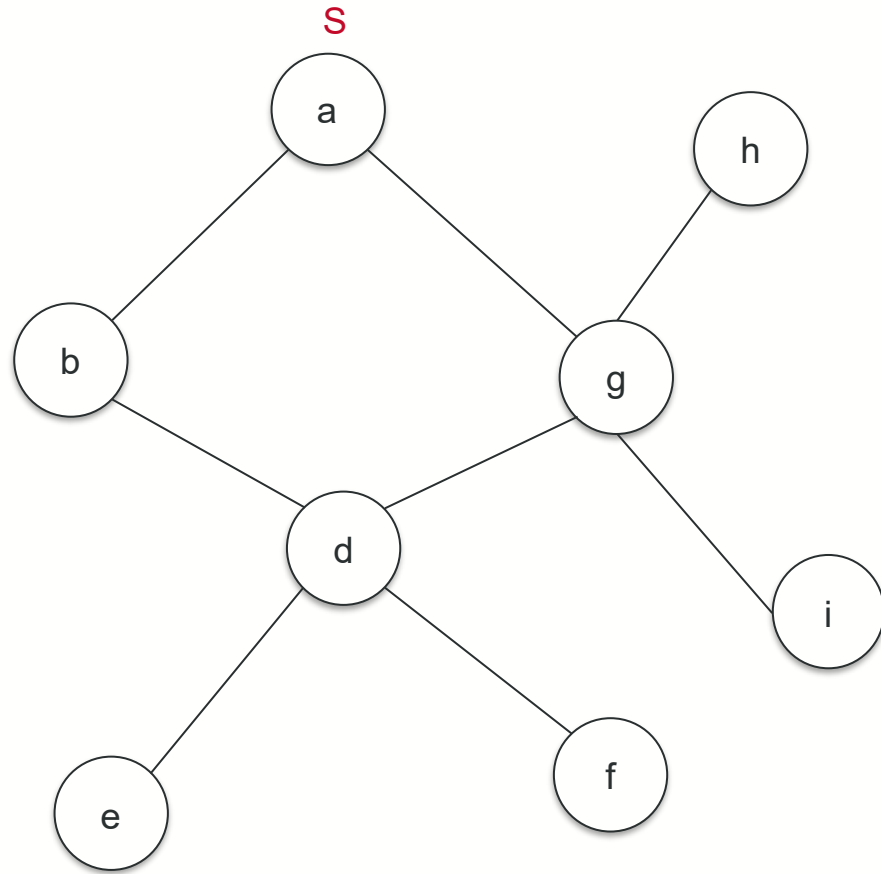
Depth-First Tree of an Undirected Graph



Depth-First Tree

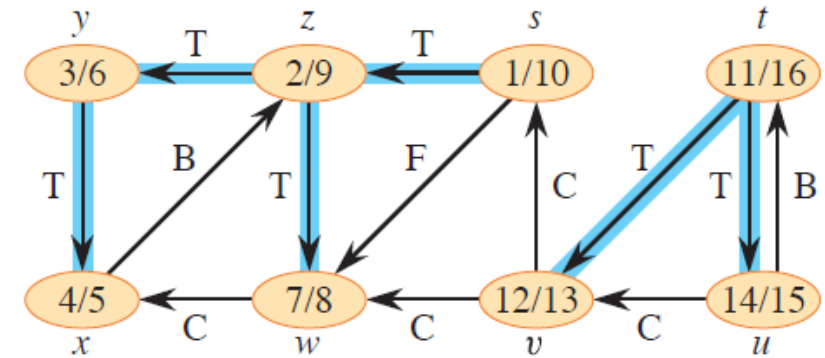


Depth-First Tree of an Undirected Graph



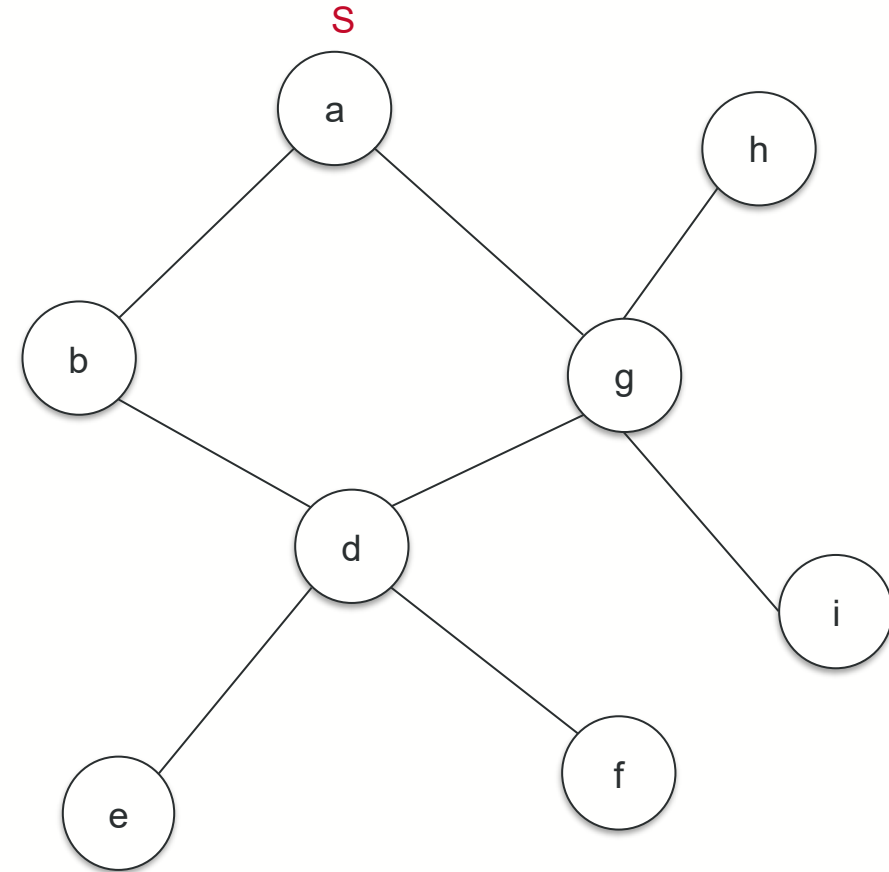
(Try it) Cycle Detection Using DFS – Directed Graph

- Identify cycles in this graph
- If DFS is exploring a node u and it sees an edge to one of its ancestors in the DFS tree, what structure does that create in the graph?
- (True/False) Every gray node encountered during DFS is an ancestor of the current node
- (True/False) An edge to a gray ancestor is always a back edge
- (True/False) A back edge in a directed graph always forms a directed cycle



Cycle Detection Using DFS – Undirected Graph

- Identify cycles in this graph
- If DFS is exploring a node u and it sees an edge to one of its ancestors in the DFS tree, what structure does that create in the graph?
- (True/False) Every gray node encountered during DFS is an ancestor of the current node
- (True/False) An edge to a gray ancestor is always a back edge
- (True/False) A back edge in a directed graph always forms a directed cycle



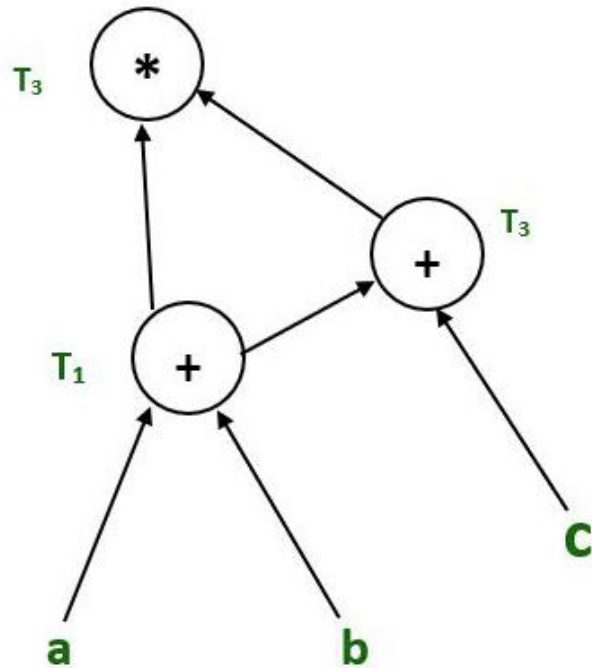
Cycle Detection Using DFS

- Directed Graph:
 - A back edge (edge to a gray node) $\rightarrow$ cycle exists
- Undirected Graph:
 - A back edge to a gray node $\rightarrow$ cycle exists, only if the gray node is not the direct parent

Topologica Sort

DAG Examples

Compiler

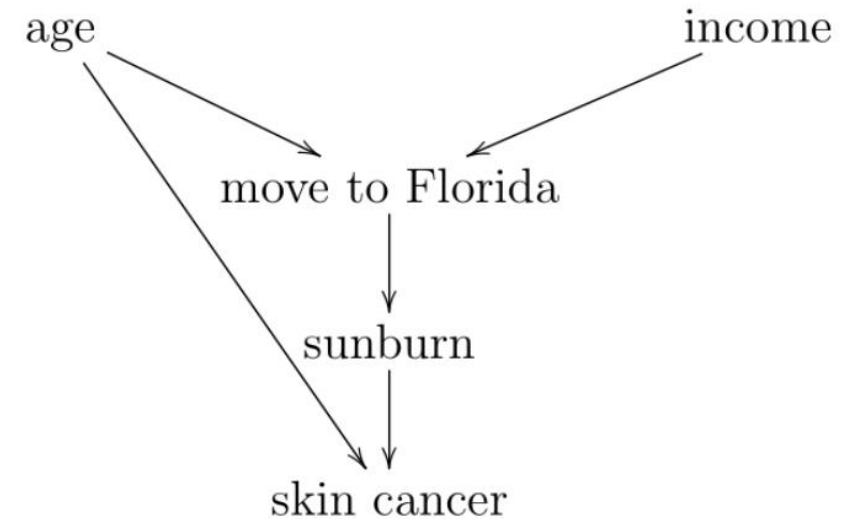


$$T_3 = T_1 * T_2$$

$$T_3 = (a+b) * (a+b+c)$$

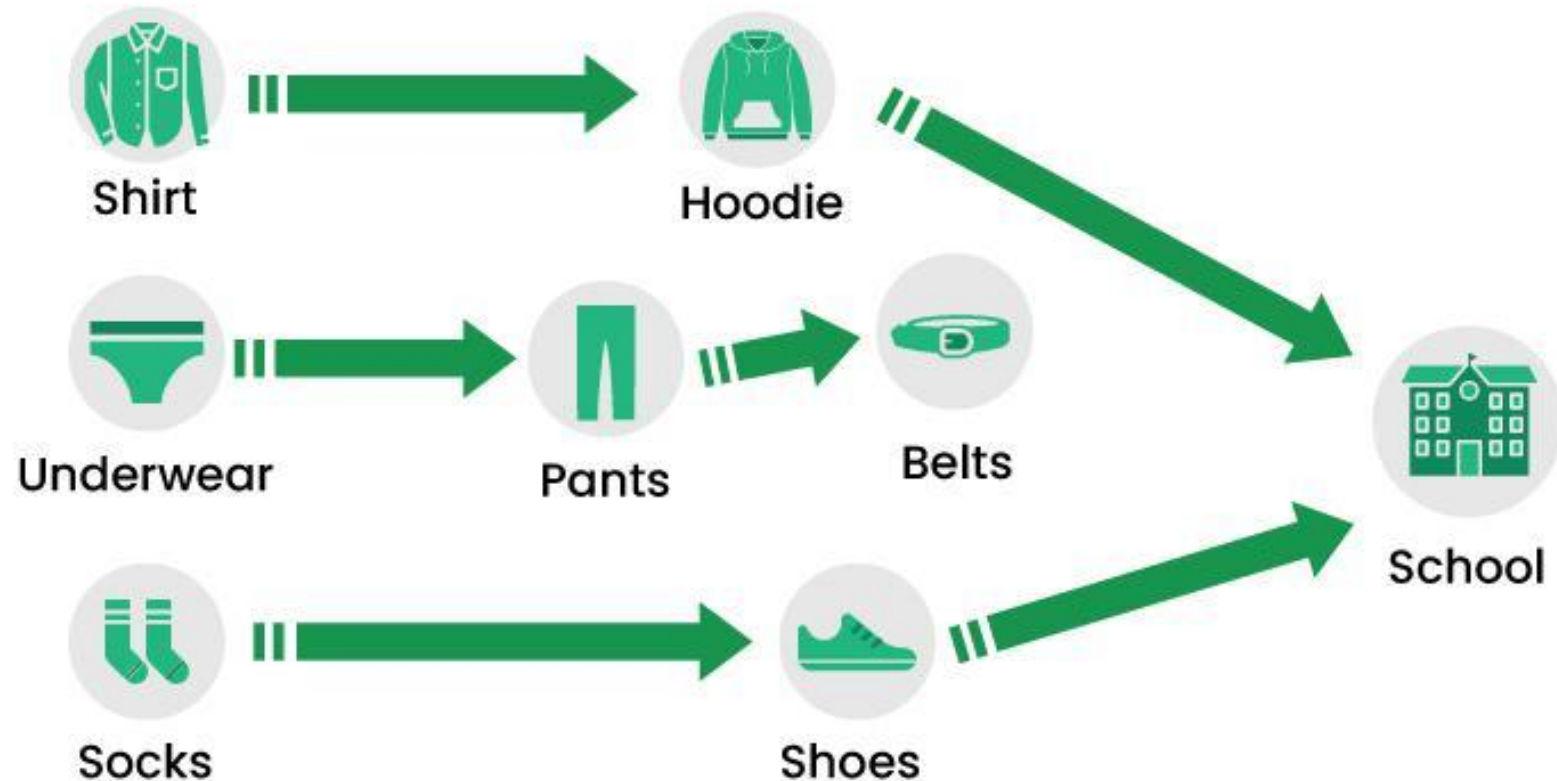
Causal relationships

Age $\rightarrow$ cancer



DAG Examples

How to get dressed



Multiple Topological ordering for a graph



Directed Acyclic Graph

- A directed acyclic graph (DAG) arises in many applications where there are precedence or ordering constraints (e.g. scheduling problems). For instance, if there are a series of tasks to be performed, and certain tasks must precede other tasks (e.g. in construction you must build the first floor before the second, but you can do the electrical wiring while you install the windows).
- In general, a precedence constraint graph is a DAG, in which vertices are tasks and edge (u, v) means that task u must be completed before task v begins.

Topological Sort

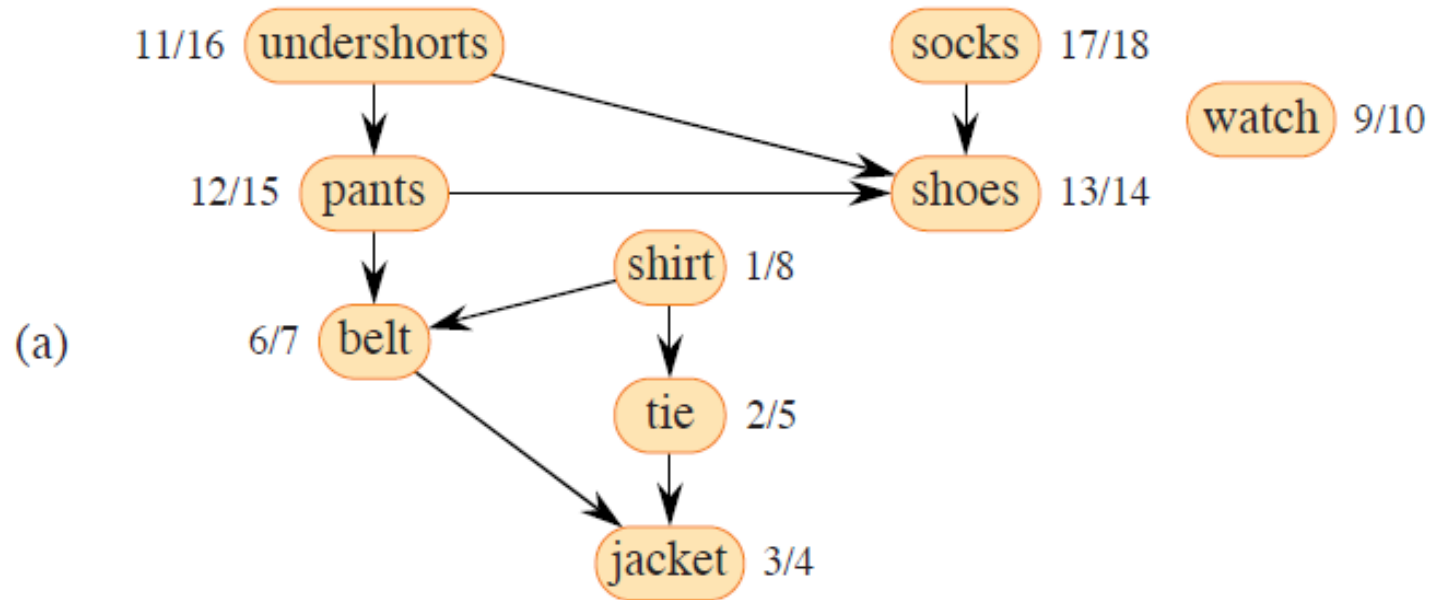
- Given a directed acyclic graph (DAG) $G = (V, E)$, topological sort is a linear ordering of all vertices of the DAG such that if G contains an edge (u, v) , u appears before v in the ordering.
- In general, there may be many legal topological orders for a given DAG.

Topological-Sort (G)

1. Call $\text{DFS}(G)$ to compute finishing time $f[v]$ for each vertex
2. As each vertex is finished, insert it onto the front of linked list
3. Return the linked list of vertices

An Example

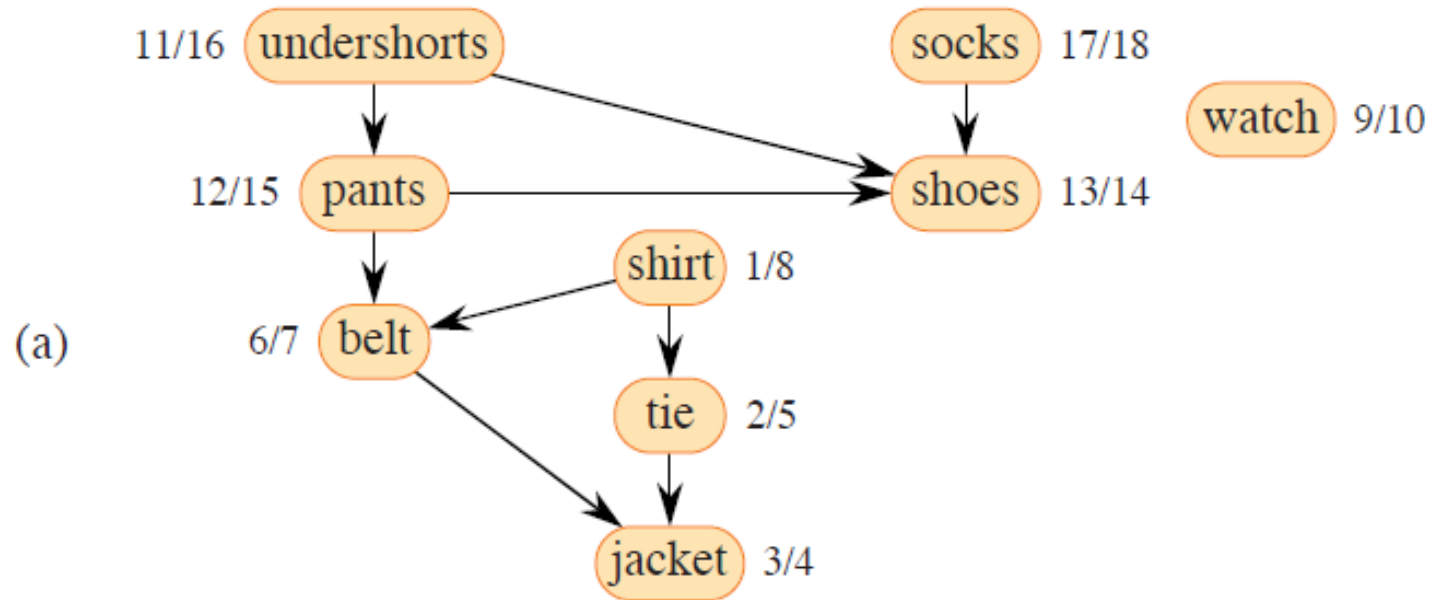
How Professor Bumstead gets dressed in the morning



The professor must don certain garments before others (e.g., socks before shoes). Other items may be put on in any order (e.g., socks and pants).

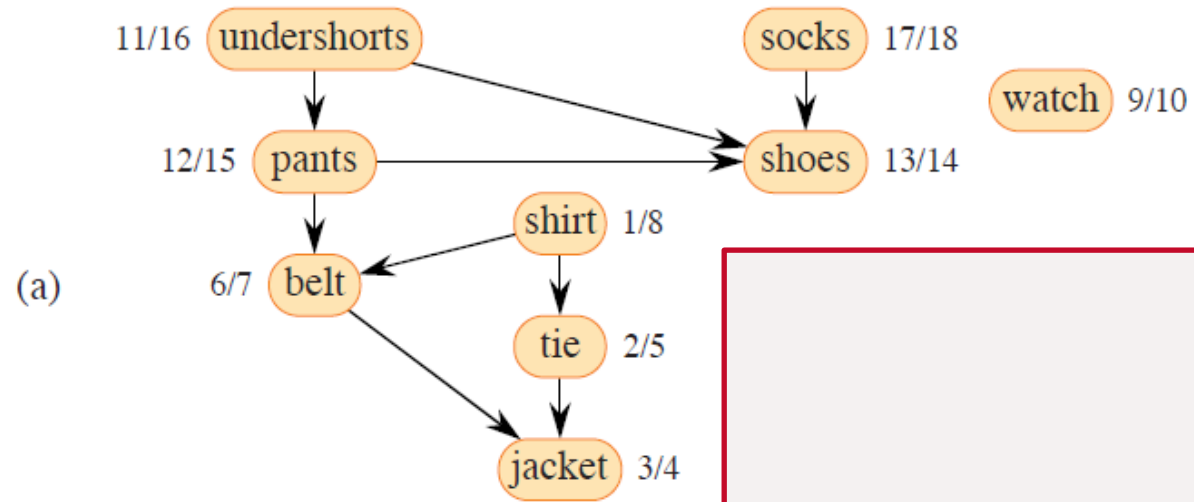
An Example: Build DFS Trees

How Professor Bumstead gets dressed in the morning

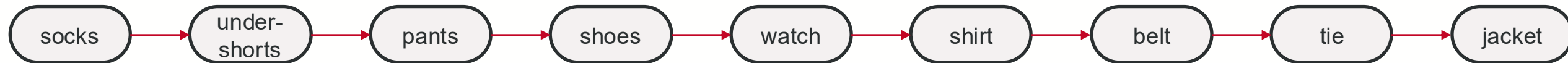
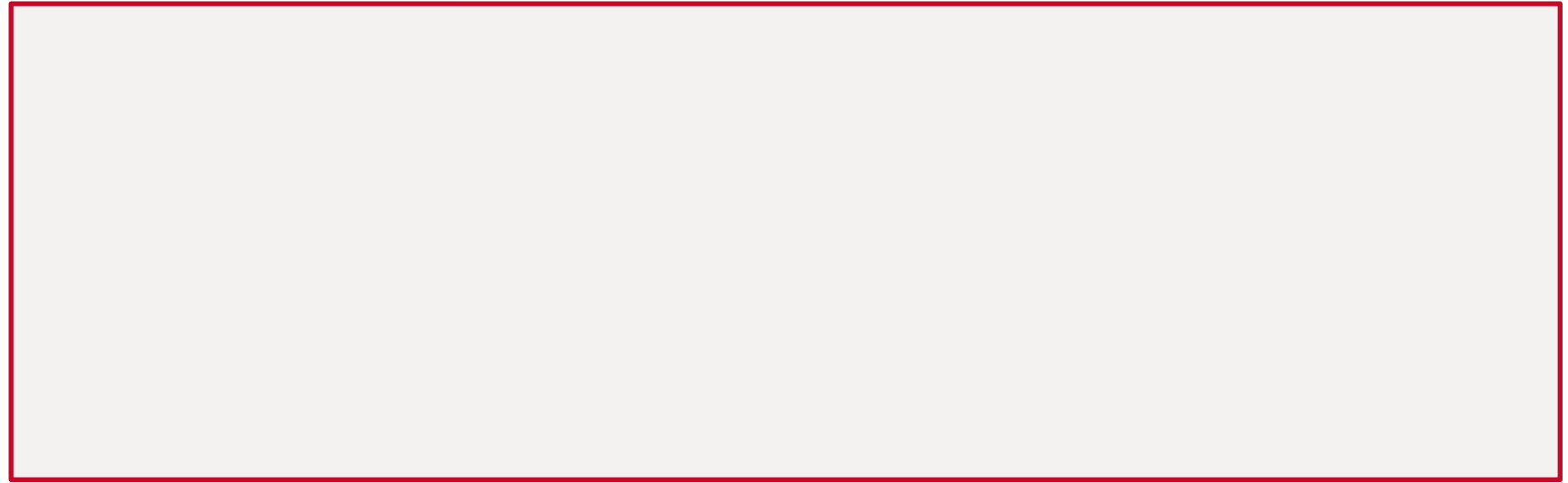


The professor must don certain garments before others (e.g., socks before shoes). Other items may be put on in any order (e.g., socks and pants).

An Example



1. Call $\text{DFS}(G)$ to compute finishing time $f[v]$ for each vertex
2. As each vertex is finished, insert it onto the front of linked list
3. Return the linked list of vertices



An Example

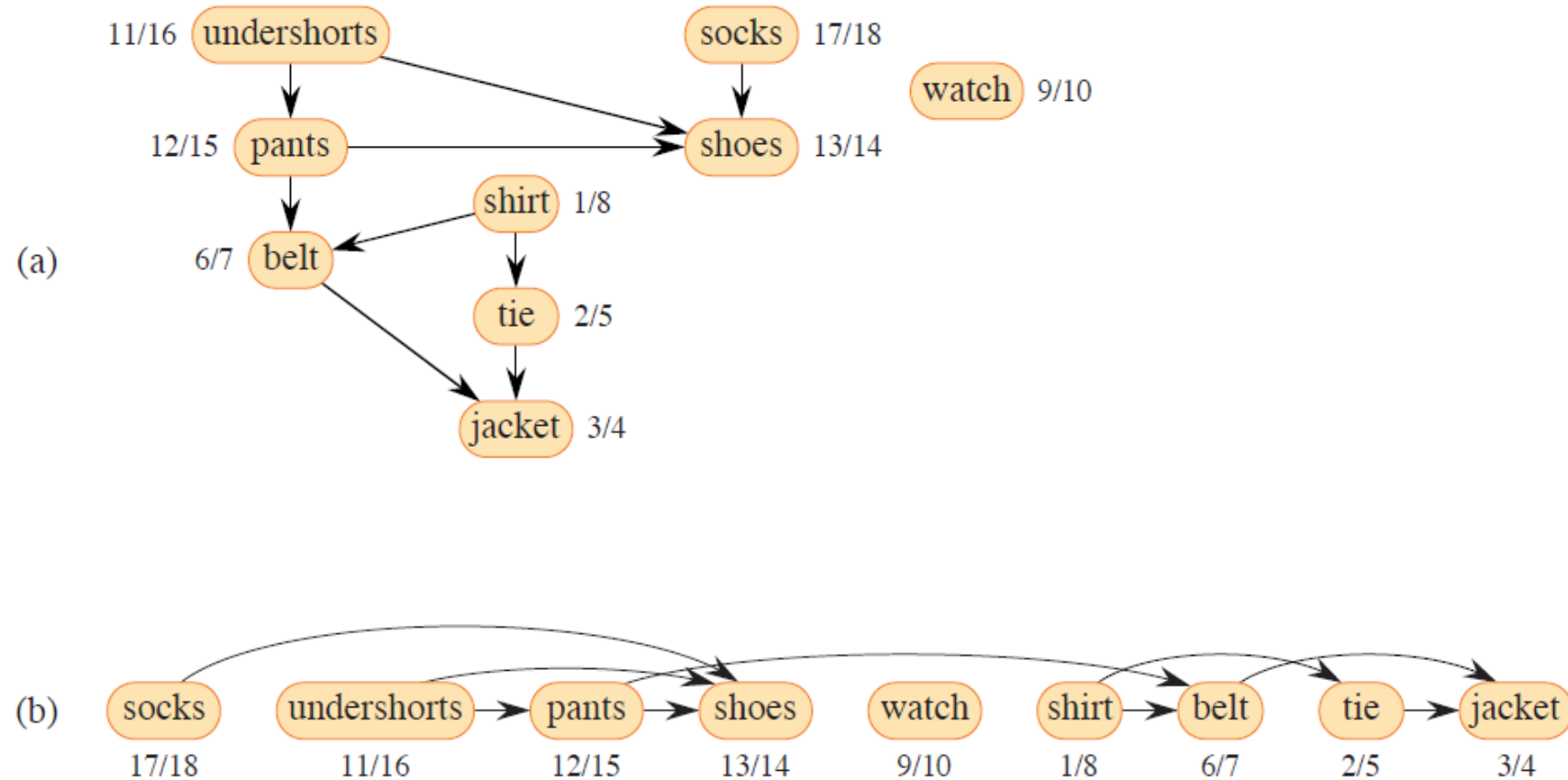
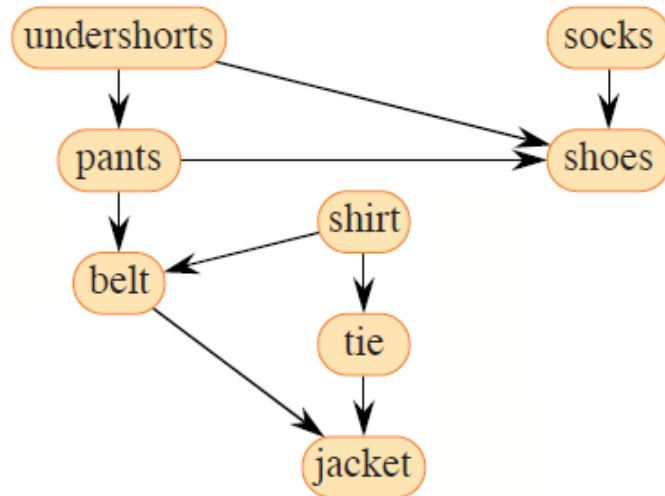


Figure 20.7 (a) Professor Bumstead topologically sorts his clothing when getting dressed. Each directed edge (u, v) means that garment u must be put on before garment v . The discovery and finish times from a depth-first search are shown next to each vertex. (b) The same graph shown topologically sorted, with its vertices arranged from left to right in order of decreasing finish time. All directed edges go from left to right.

Another Example

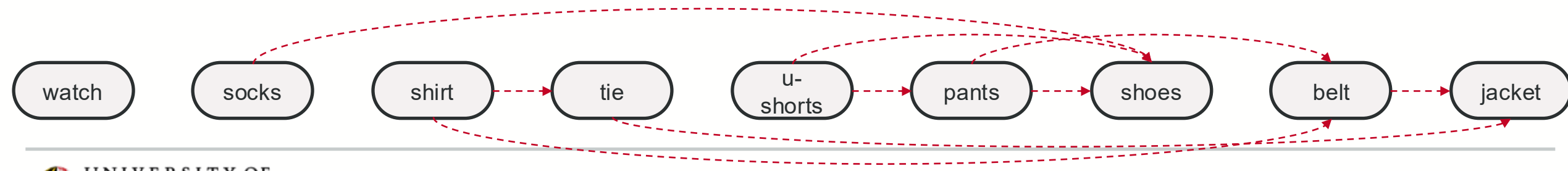
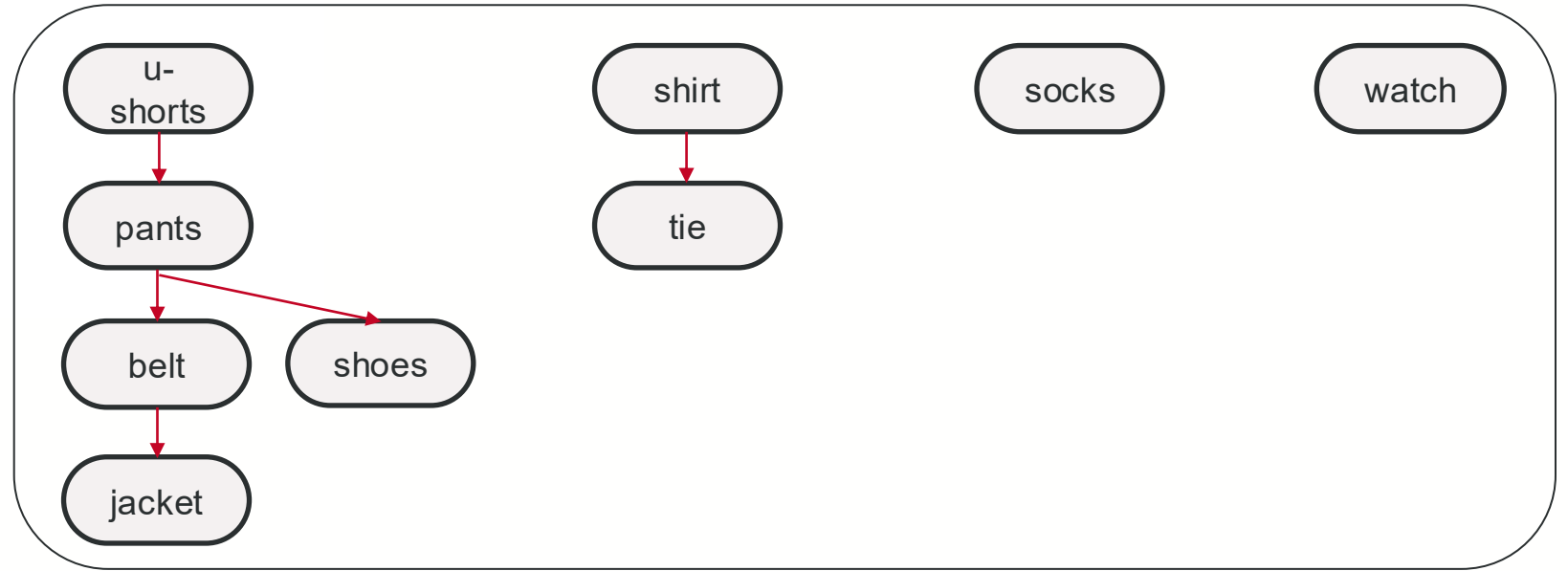
- Consider the same example, but we do our DFS in a different order, so we get a different final ordering. But, both are legitimate, given the precedence constraints.

Another Example (1)

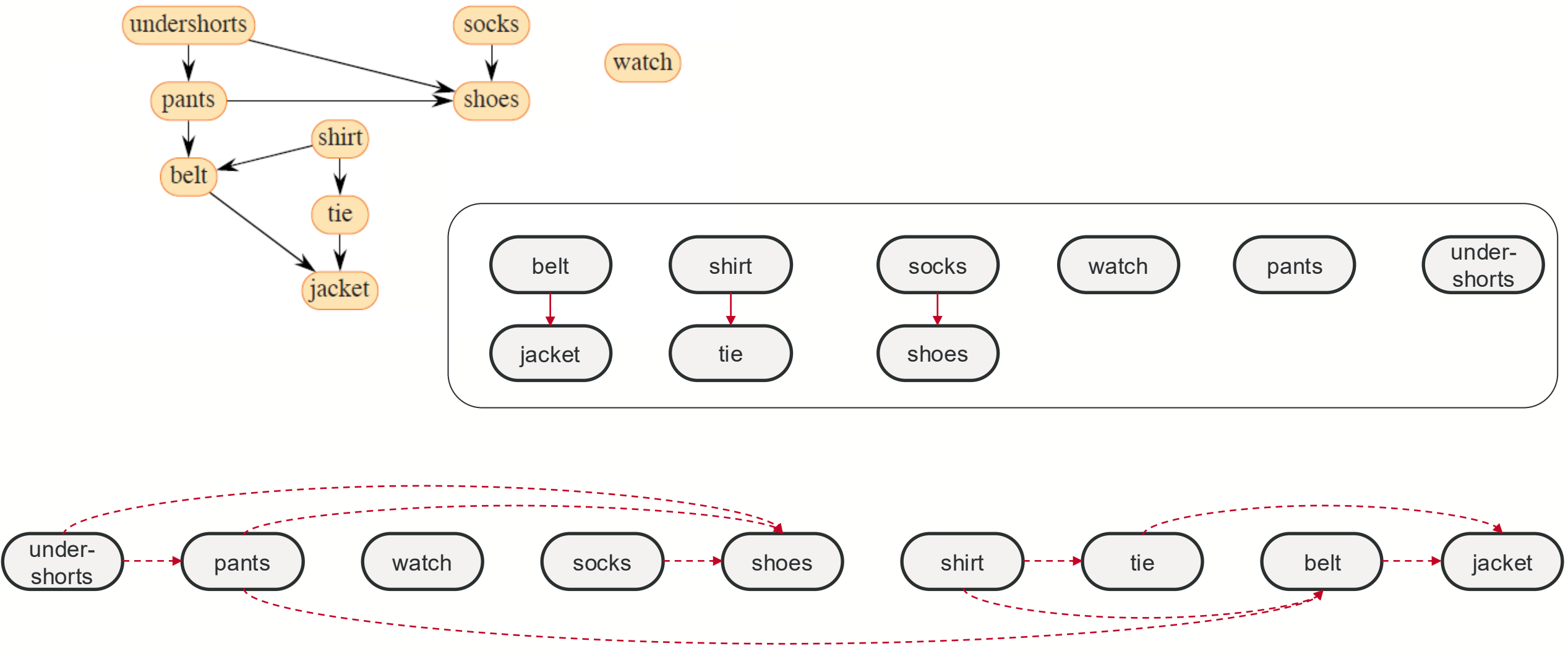


watch

1. Call $\text{DFS}(G)$ to compute finishing time $f[v]$ for each vertex
2. As each vertex is finished, insert it onto the front of linked list
3. Return the linked list of vertices



Another Example (2)



Recall - DFS(G)

DFS(G)

1. for each vertex $u \in V[G]$
2. do $color[u] \leftarrow \text{WHITE}$
3. $\pi[u] \leftarrow \text{NIL}$
4. $time \leftarrow 0$
5. for each vertex $u \in V[G]$
6. do if $color[v] = \text{WHITE}$
7. then DFS-Visit(v)

DFS-Visit(v)

1. $color[u] \leftarrow \text{GRAY}$
 ∇ White vertex u has been discovered
2. $d[u] \leftarrow ++time$
3. for each vertex $v \in Adj[u]$
4. do if $color[v] = \text{WHITE}$
5. then $\pi[v] \leftarrow u$
6. DFS-Visit(v)
7. $color[u] \leftarrow \text{BLACK}$
 ∇ Blacken u ; it is finished.
8. $f[u] \leftarrow time++$

A Compact Version: TopSort(G)

```
1. for each  $u \in V$  color[ $u$ ] = white;           // initialization
2.  $L$  = new linked_list;                          //  $L$ : empty linked list
3. for each  $u \in V$ 
4.     if (color[ $u$ ] == white) TopVisit( $u$ );
5. return  $L$ ;                                     //  $L$  gives final order

TopVisit( $u$ ) {                                     // start search at  $u$ 
1. color[ $u$ ] = gray;                             // mark  $u$  visited
2. for each  $v \in \text{Adj}(u)$ 
3.     if (color[ $v$ ] == white) TopVisit( $v$ );
4. append  $u$  to the front of  $L$                   // finish & add  $u$  to  $L$ 
}
```

Topological Sort Visualization

Topological Sort (DFS)

Do Topological Sort

New Graph

- ☒ Small Graph ☒ Logical Representation
☐ Large Graph ☐ Adjacency List Representation
☐ Adjacency Matrix Representation

DFS(0)

DFS(1)

DFS(2)

DFS(4)

DFS(5)

DFS(7)

DFS(6)

DFS(3)

Topological Order

0

3

1

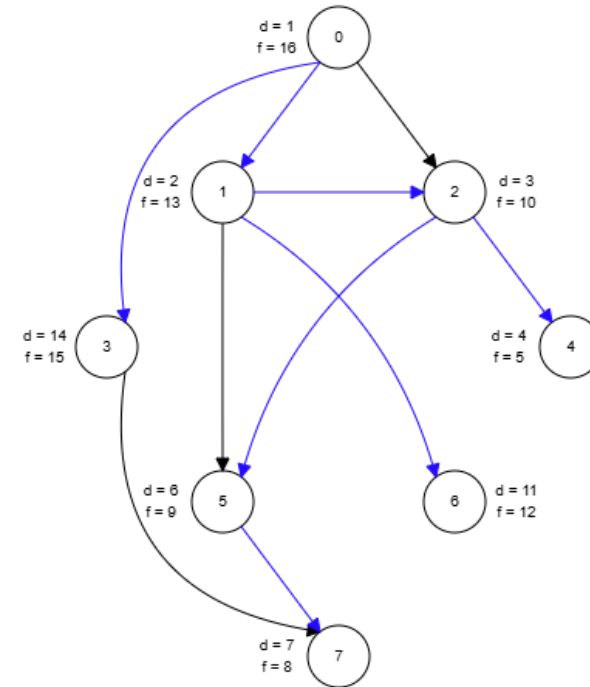
6

2

5

7

4



Animation Completed

<https://www.cs.usfca.edu/~galles/visualization/TopoSortDFS.html>

Analysis of Topological-Sort

- Line1: DFS takes $\Theta(V+E)$
- Line 2: Each insertion takes $O(1)$ for $|V|$ vertices
- Total: $\Theta(V+E)$ time

Strongly Connected Components

(In directed graphs)

Lemma

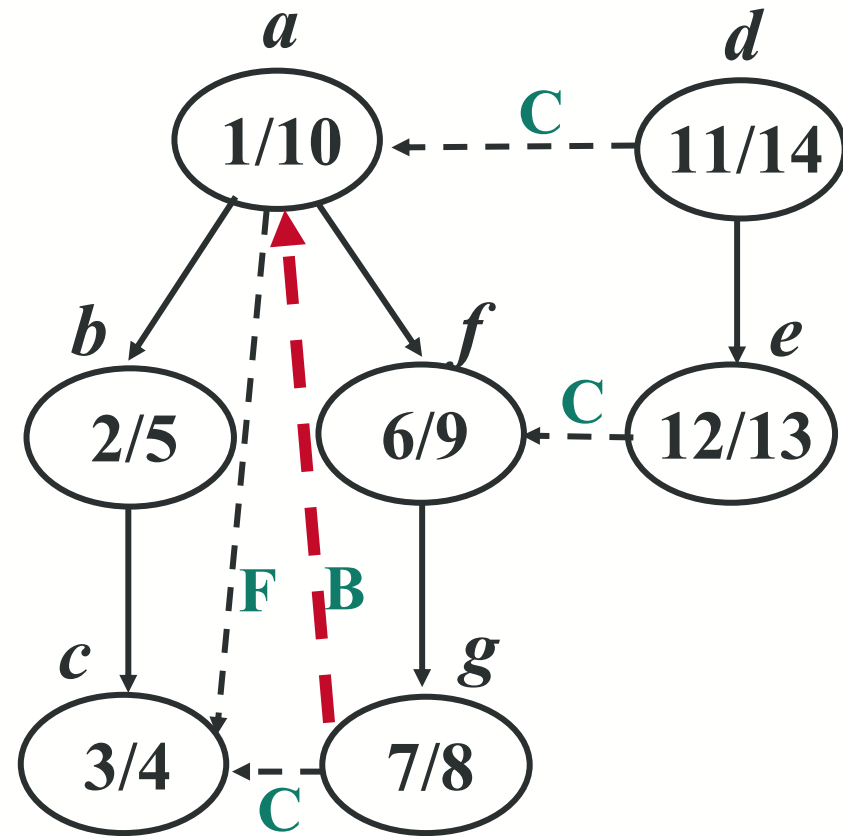
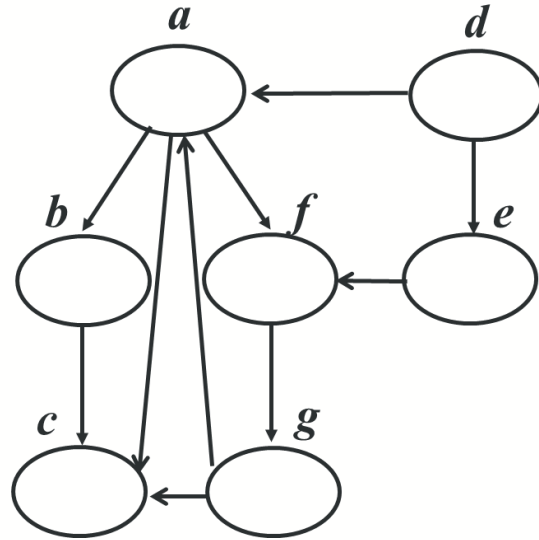
- Consider a directed graph G and a forest of DFS for G . G has a cycle if and only if the DFS forest has a back edge.

Proof:

- ($\Leftarrow$) If there is a back edge (u, v) , then v is an ancestor of u , and by following tree edges from v to u , we get a cycle.
- ($\Rightarrow$) By contrapositive: suppose there are no back edges. By White-Path Theorem, each of the remaining types of edges, tree, forward and cross all have the property that they go from vertices with higher finishing time to vertices with lower finishing time. Thus, along any path, finish time monotonically decrease. This implies there can be no cycle.

Proof ($\Leftarrow$)

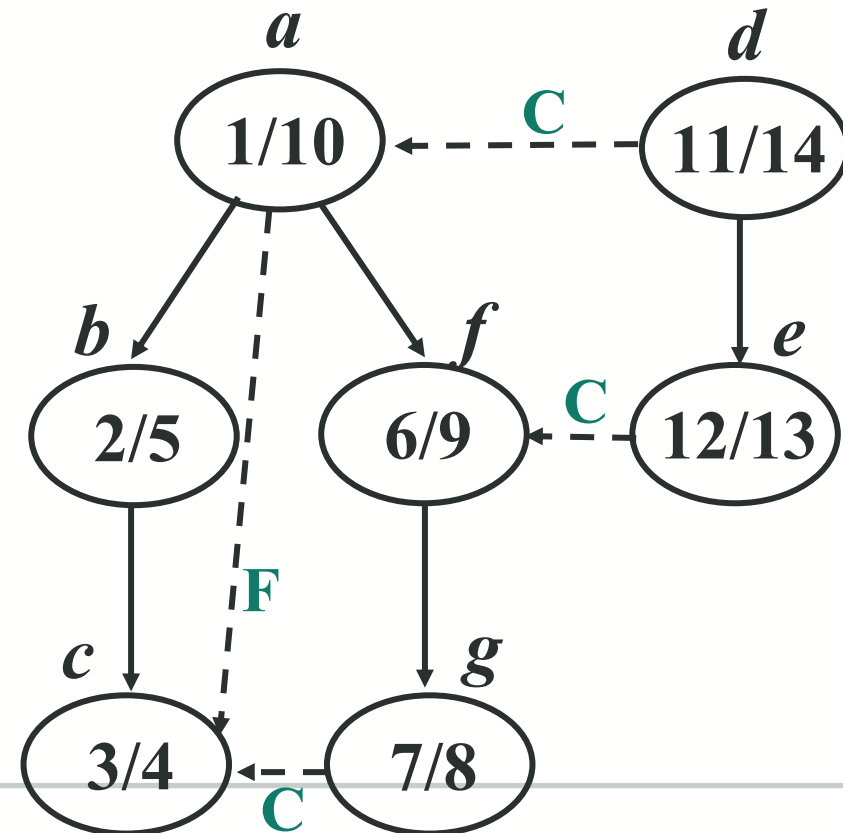
- If there is a back edge (u, v) , then v is an ancestor of u , and by following tree edges from v to u , we get a cycle.



Proof ($\Rightarrow$)

- By contrapositive: suppose there are no back edges. By White-Path Theorem, each of the remaining types of edges, tree, forward and cross all have the property that they go from vertices with higher finishing time to vertices with lower finishing time. Thus, along any path, finish time monotonically decrease. This implies there can be no cycle.

- If finish times **always decrease**, we can never return to a previously visited node in a way that would create a cycle.
- The graph must be a DAG

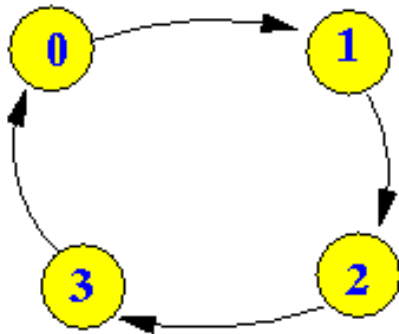


Applications of DFS

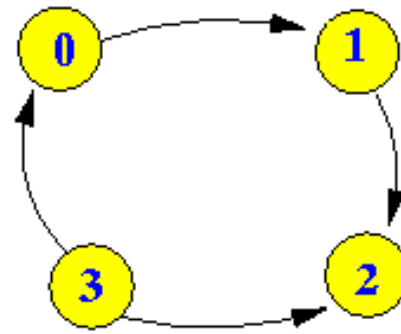
- Decomposing a directed graph into its strongly connected components
- Determining articulation points, bridges & biconnected components of an undirected graph

Strongly Connected Components

- Digraphs are often used in communication and transportation networks, where people want to know that the networks are complete in the sense that from any location it is possible to reach another location in the digraph.
- A digraph is “*strongly connected*”, if for every pair of vertices $u, v \in V$, u can reach v and vice versa. We say that two vertices u and v are “*mutually reachable*”. Mutual reachability is an equivalence relation.



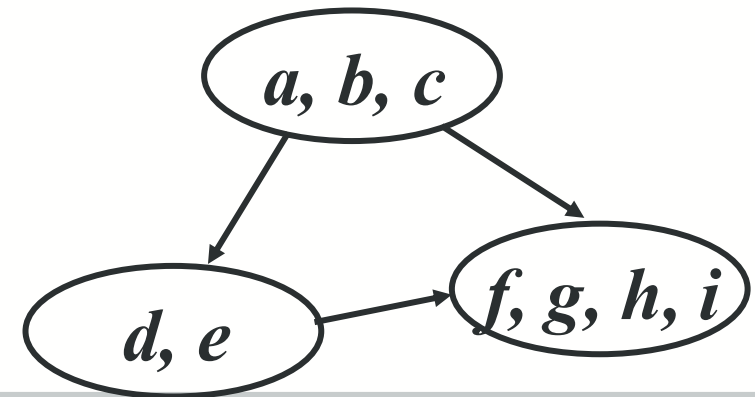
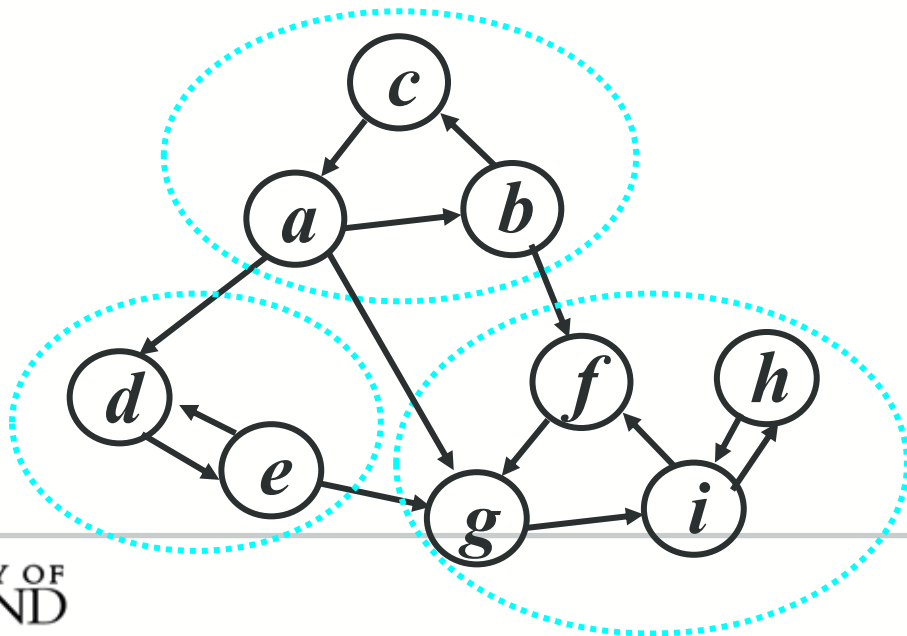
Strongly connected



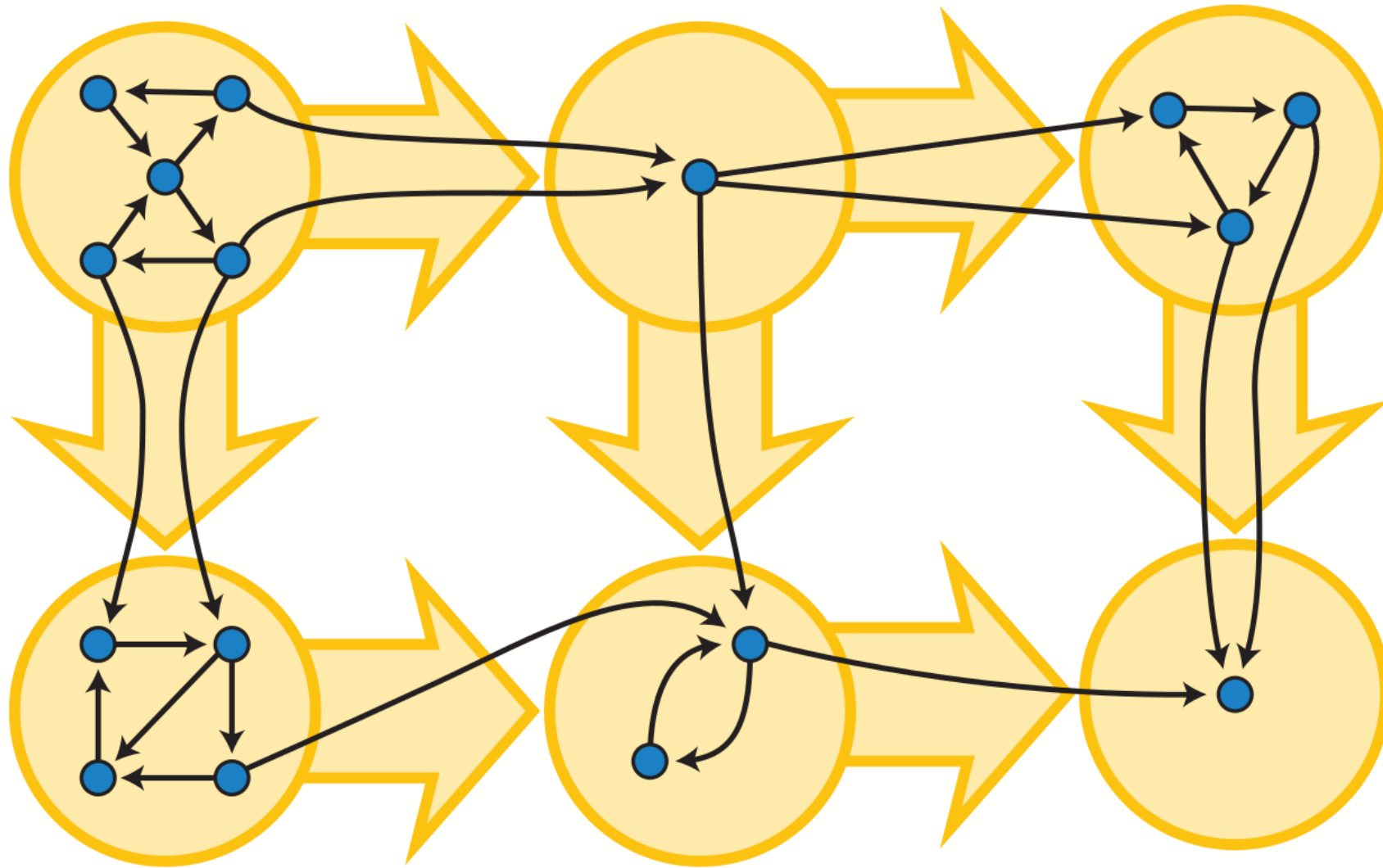
Not strongly connected

Component DAG

- If we merge the vertices in each strong component into a single *super vertex*, and joint two super vertices (A, B) if and only if there are vertices $u \in A$ and $v \in B$ such that $(u, v) \in E$, then the resulting digraph, called the *component digraph*, is necessarily acyclic.



Component DAG

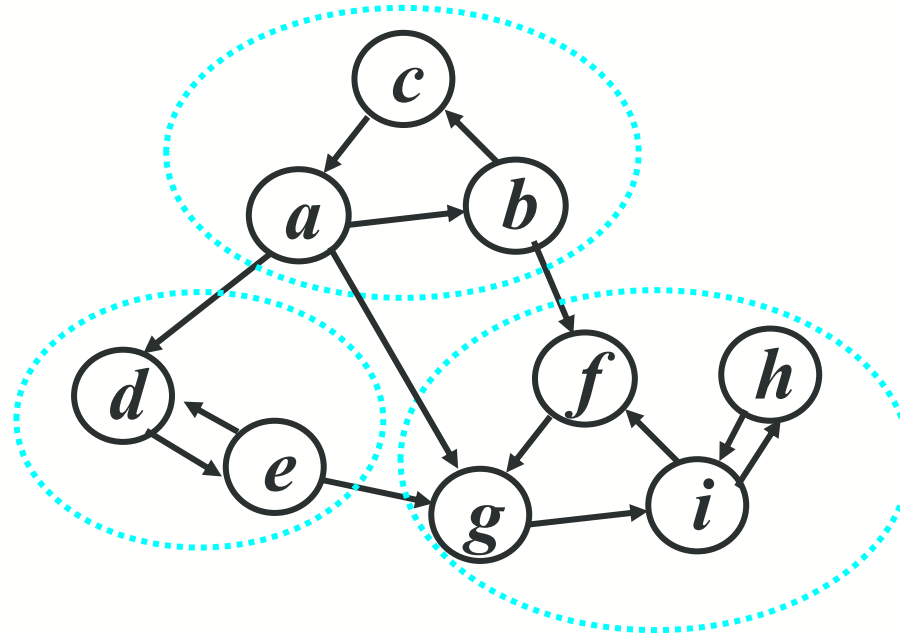


Finding Strongly Connected Components

- Use depth-first search twice
- Trick: Reverse directions of edges, search in reverse order of finishing times (topological order for reversed graph)
- But why do we need this trick...?

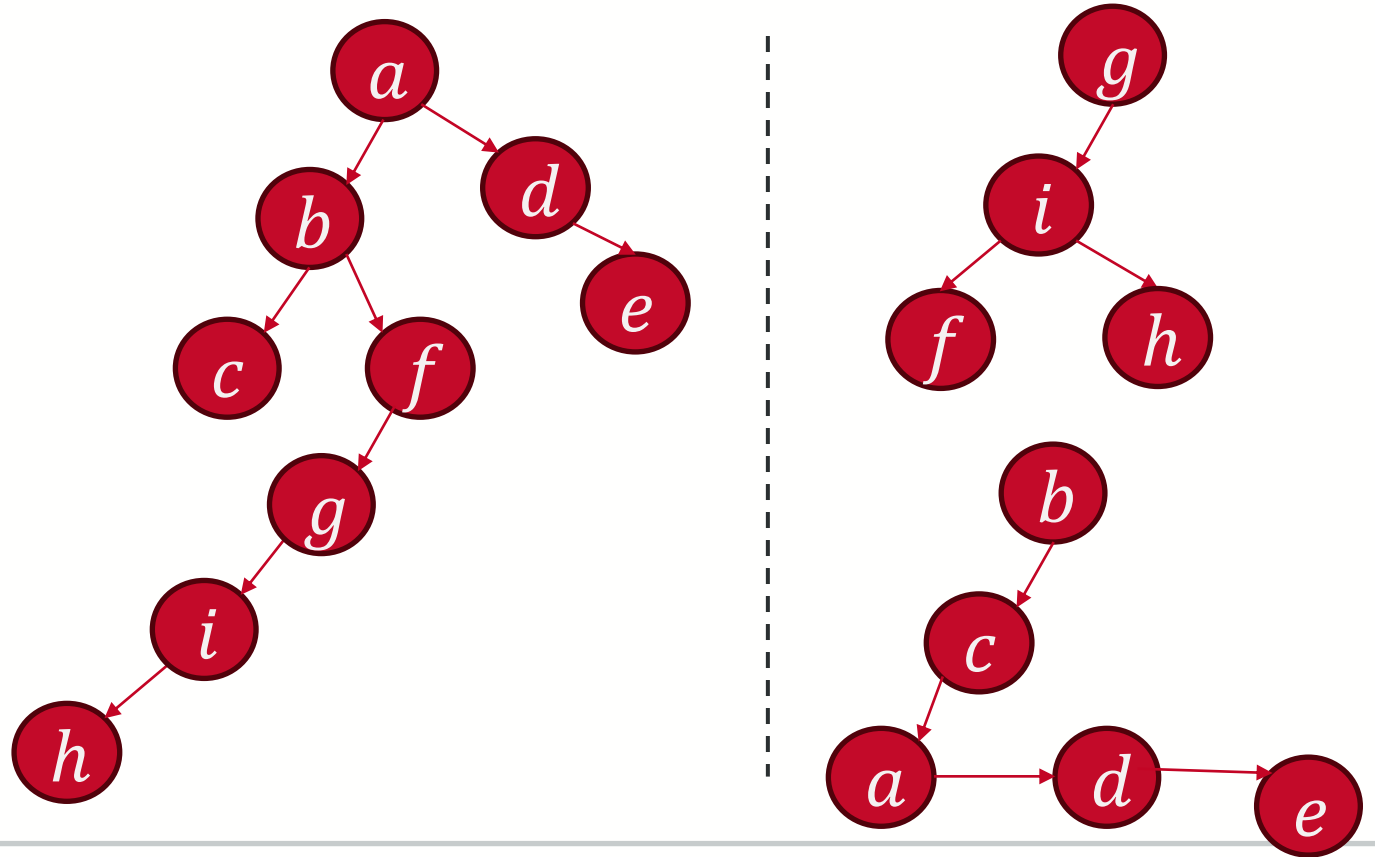
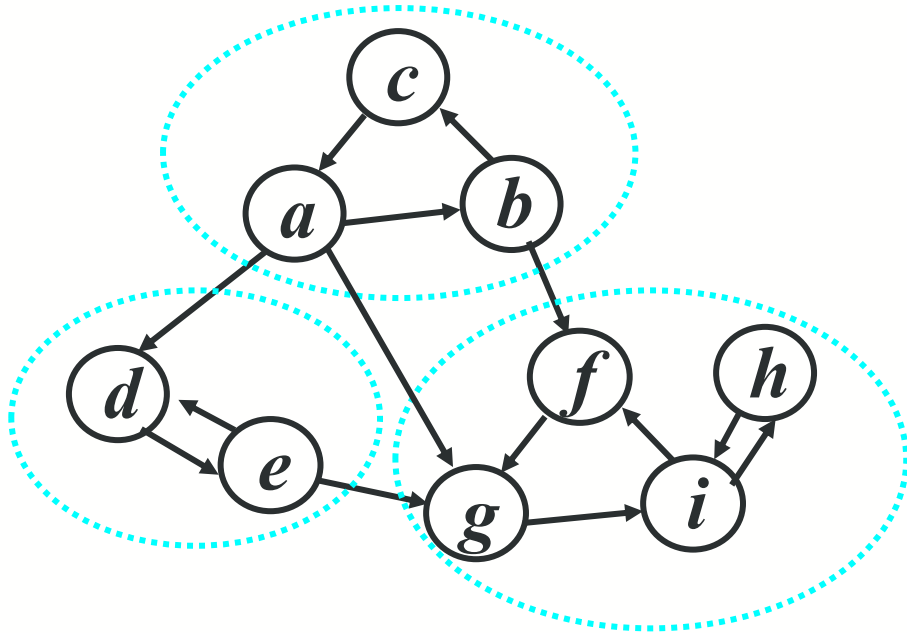
Strong Components & DFS

- Each strong component is just a subtree of DFS Forest.
- Does there always exist a way to order the DFS such that it is true?



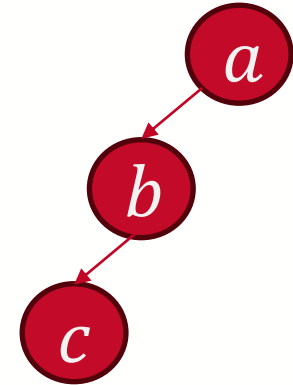
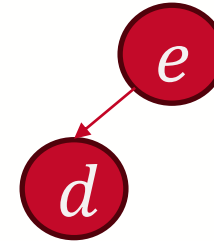
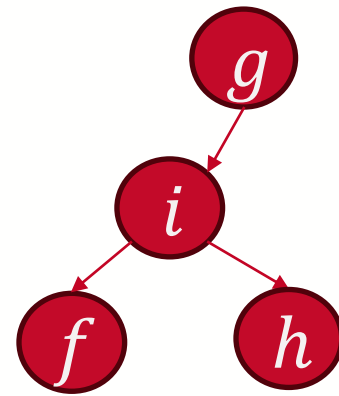
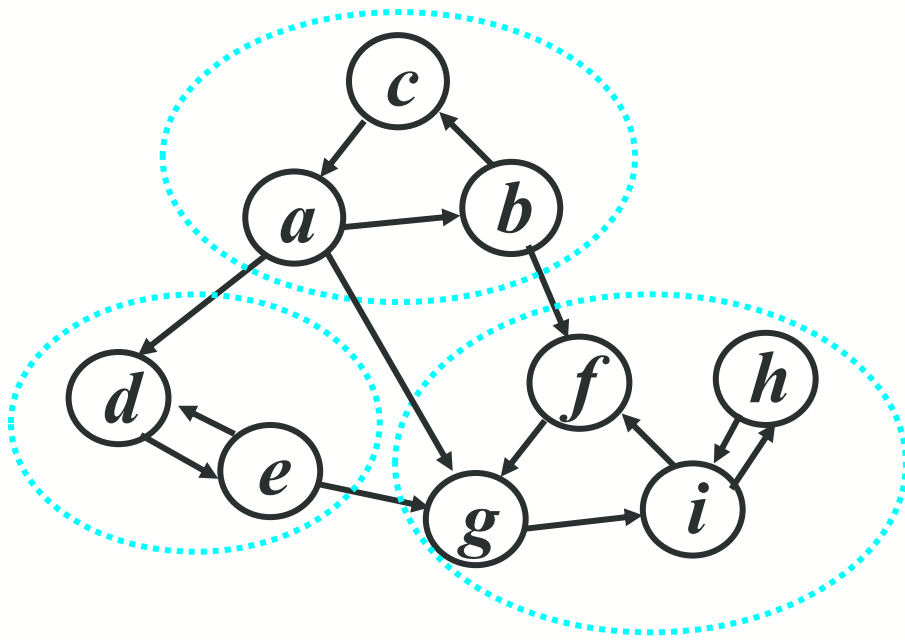
Strong Components & DFS

- Each strong component is just a subtree of DFS Forest.
- Does there always exist a way to order the DFS such that it is true?



Strong Components & DFS

- Each strong component is just a subtree of DFS Forest.
- Does there always exist a way to order the DFS such that it is true?



Ordering DFS

- Once the DFS starts within a given strong component, it must visit every vertex within the component (and possibly some others) before finishing.
- The search may “leak out” into other strong components, so doesn’t give us a list of strong components.
- But, if we visit nodes in reverse topological order, each search cannot leak out into other components, since those components would have been visited earlier in the search.

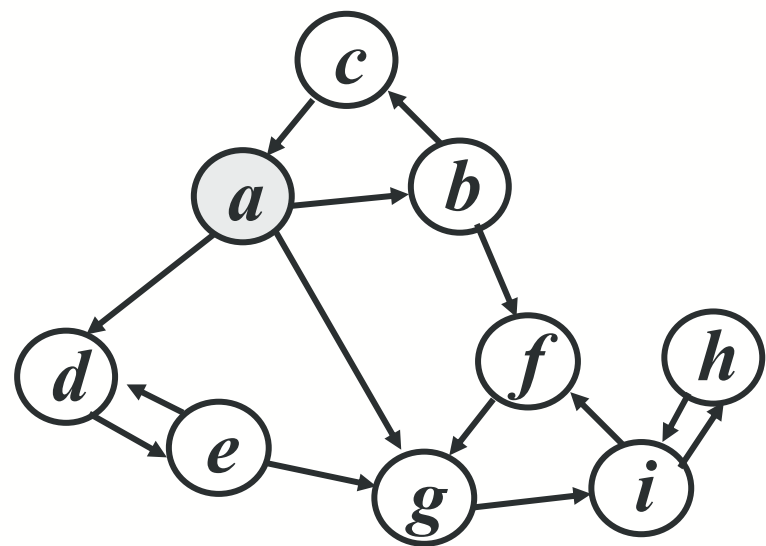
StrongComp(G)- Kosaraju's Algorithm

1. Run DFS(G), computing finish time $f[u]$ for each vertex u
2. Compute $R = \text{Reverse}(G)$, reversing all edges of G
3. Sort the vertices of R (by CountingSort) in decreasing order of $f[u]$
4. Run DFS(R) using this order
5. Each DFS tree is a strong component; output vertices of each tree in the DFS forest

Total running time is $\Theta(V + E)$

Counting Sort $\Theta(V)$ + Two DFS traversals $\Theta(V + E)$

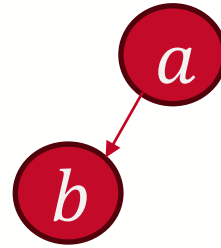
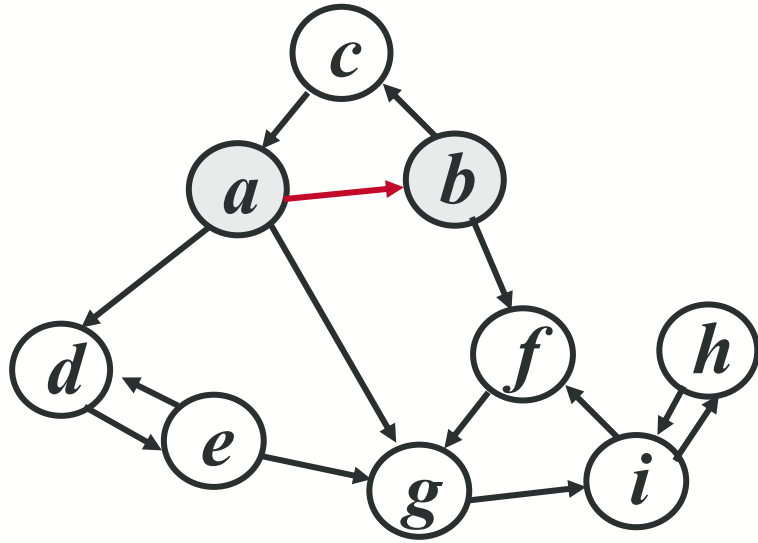
Strong Components & DFS



a

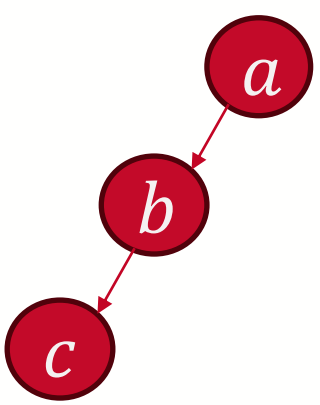
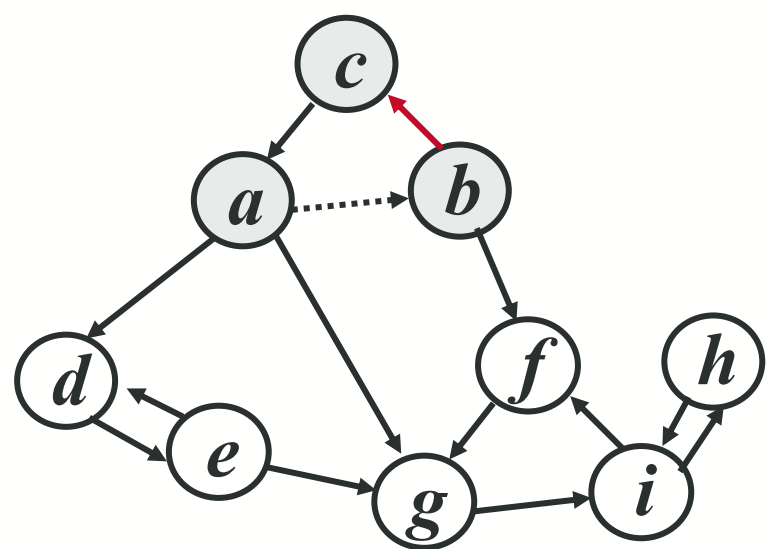
<i>v</i>	$\pi[v]$	$d[v]$	$f[v]$
<i>a</i>	<i>NIL</i>	<i>1</i>	
<i>b</i>			
<i>c</i>			
<i>d</i>			
<i>e</i>			
<i>f</i>			
<i>g</i>			
<i>h</i>			
<i>i</i>			

Strong Components & DFS



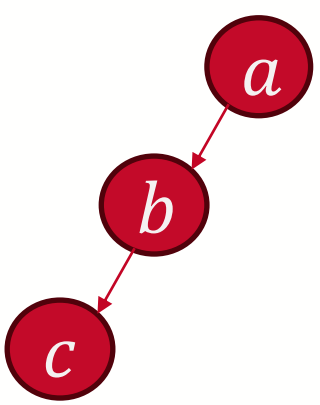
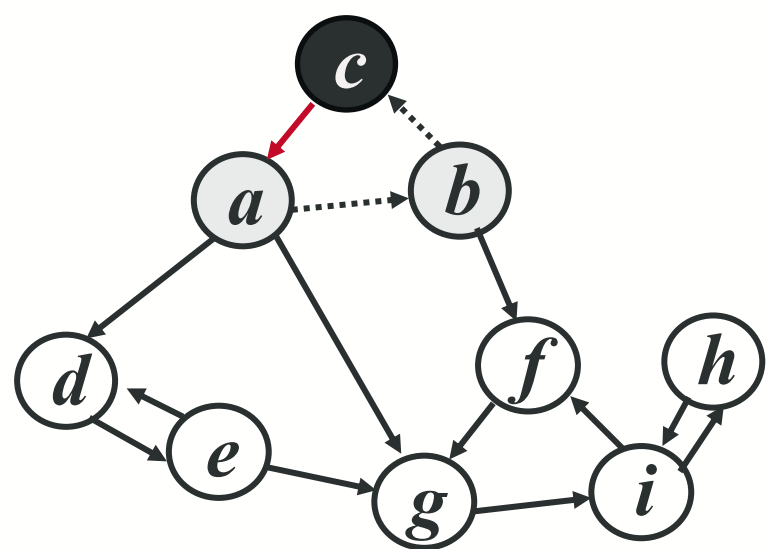
v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	
b	a	2	
c			
d			
e			
f			
g			
h			
i			

Strong Components & DFS



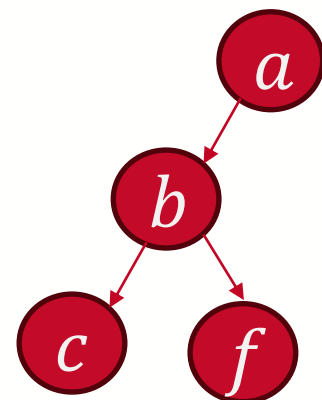
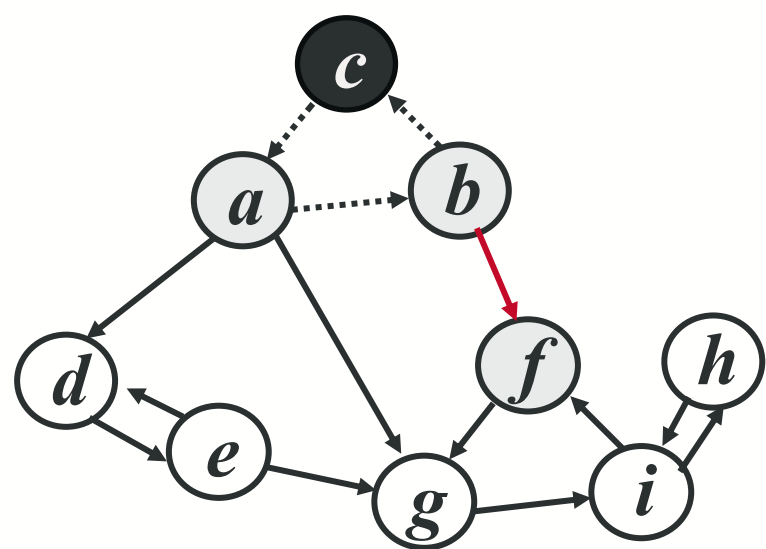
v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	
b	a	2	
c	b	3	
d			
e			
f			
g			
h			
i			

Strong Components & DFS



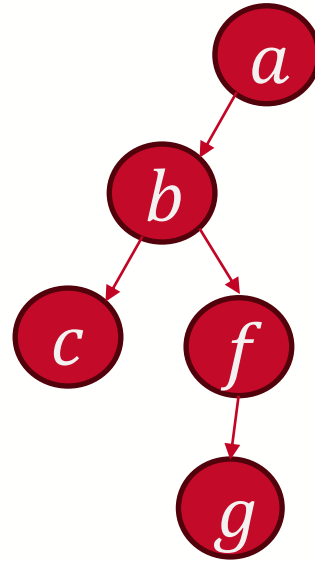
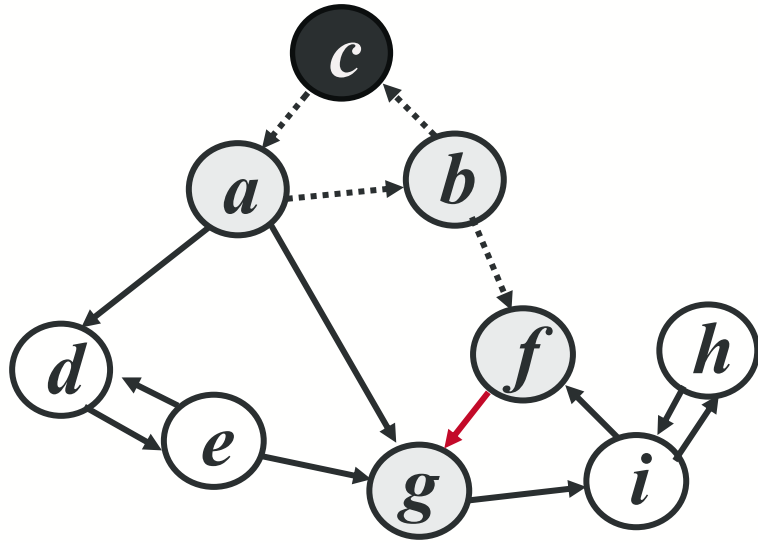
v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	
b	a	2	
c	b	3	4
d			
e			
f			
g			
h			
i			

Strong Components & DFS



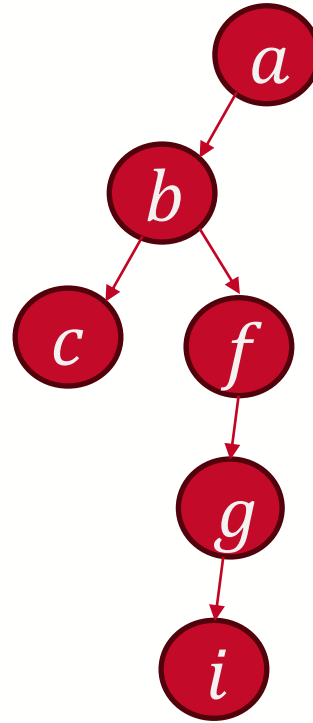
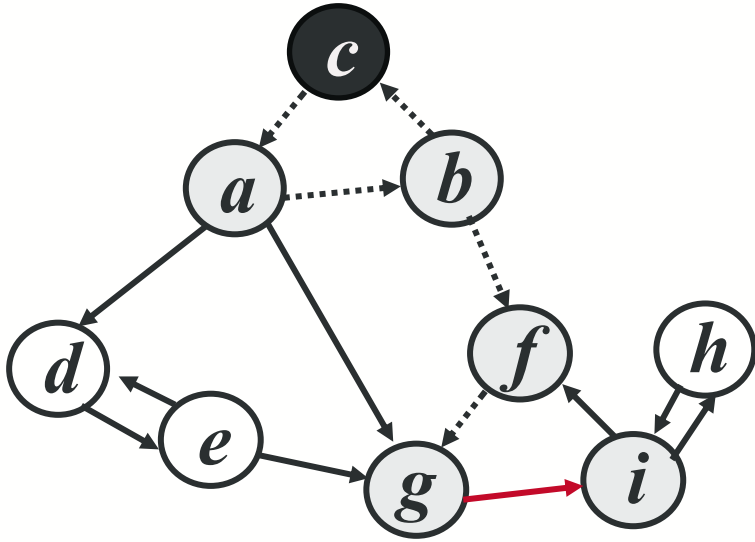
v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	
b	a	2	
c	b	3	4
d			
e			
f	b	5	
g			
h			
i			

Strong Components & DFS



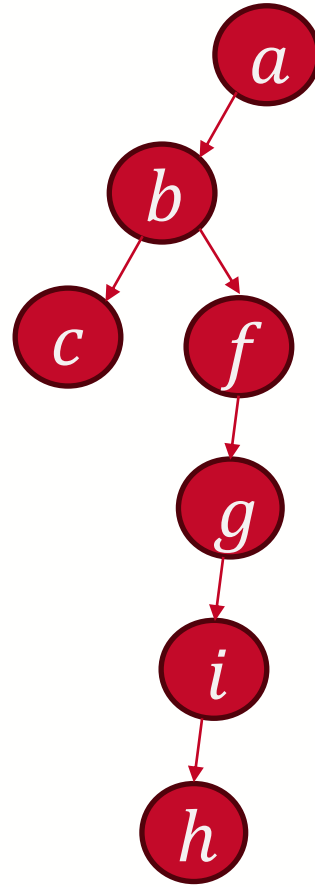
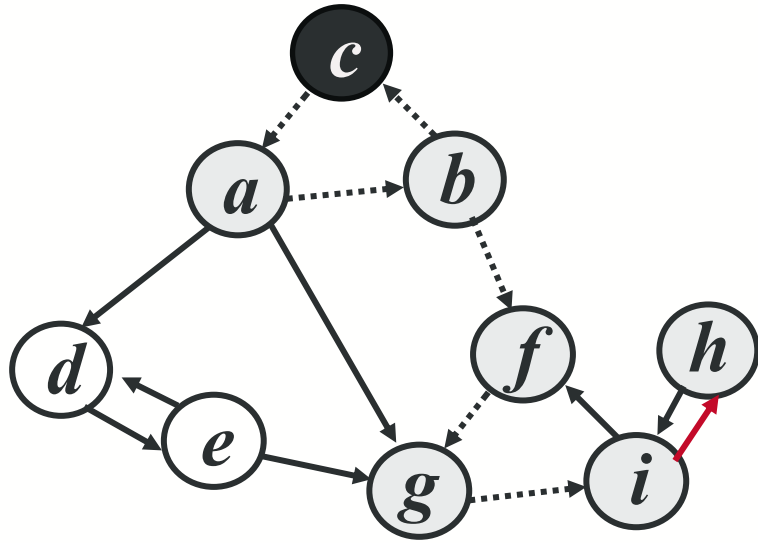
v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	
b	a	2	
c	b	3	4
d			
e			
f	b	5	
g	f	6	
h			
i			

Strong Components & DFS



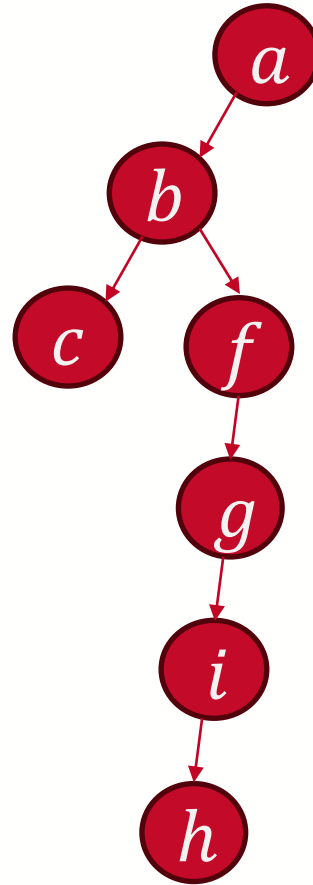
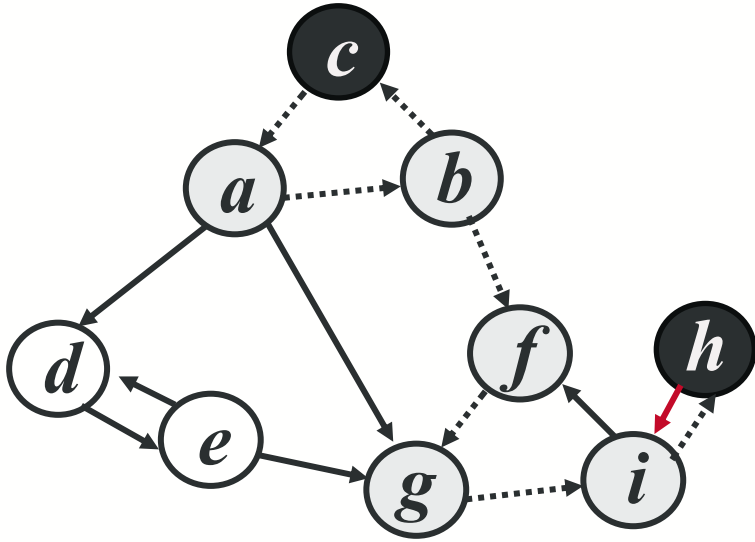
v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	
b	a	2	
c	b	3	4
d			
e			
f	b	5	
g	f	6	
h			
i	g	7	

Strong Components & DFS



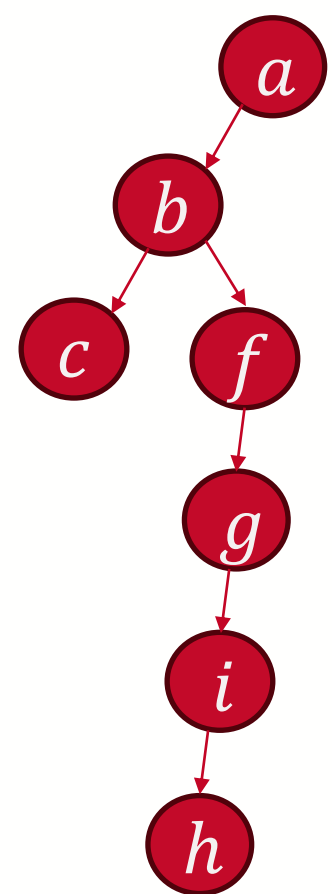
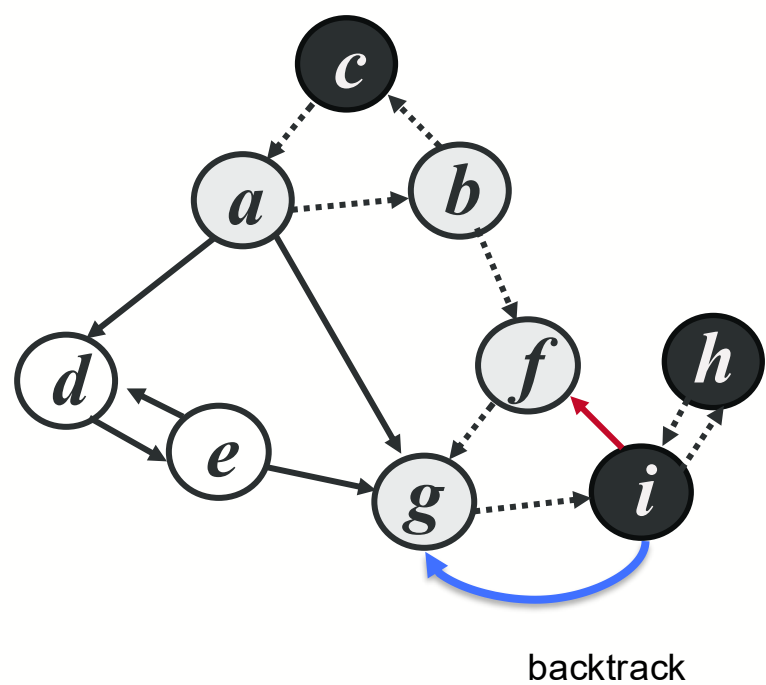
v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	
b	a	2	
c	b	3	4
d			
e			
f	b	5	
g	f	6	
h	i	8	
i	g	7	

Strong Components & DFS



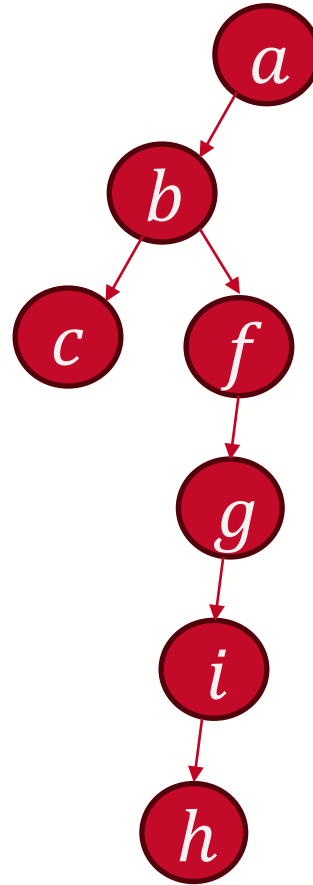
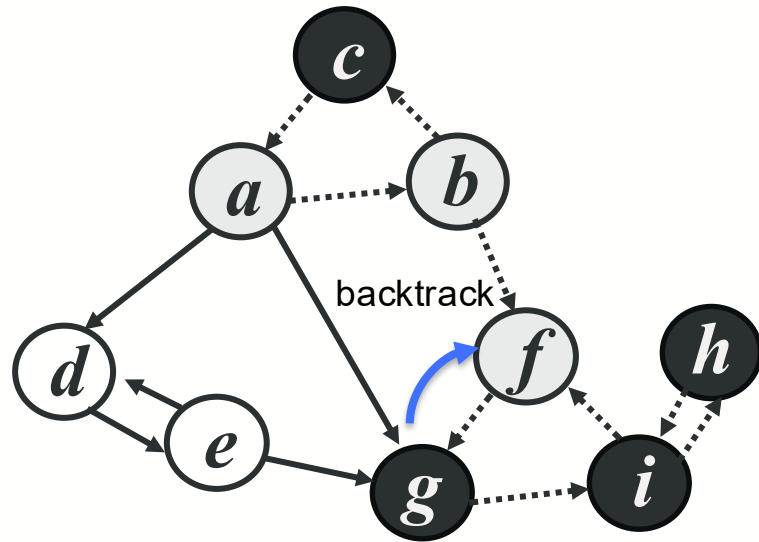
v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	
b	a	2	
c	b	3	4
d			
e			
f	b	5	
g	f	6	
h	i	8	9
i	g	7	

Strong Components & DFS



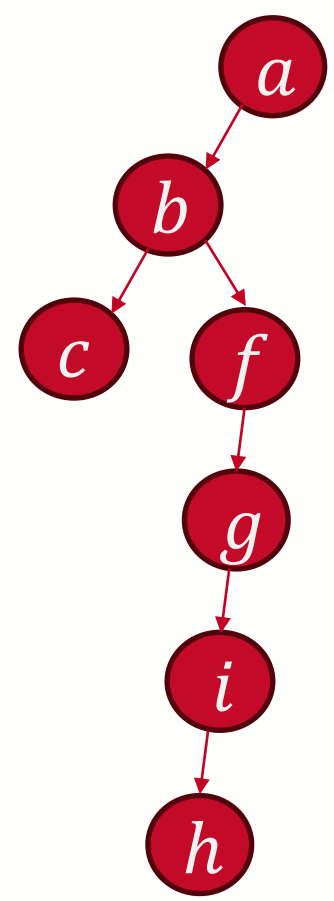
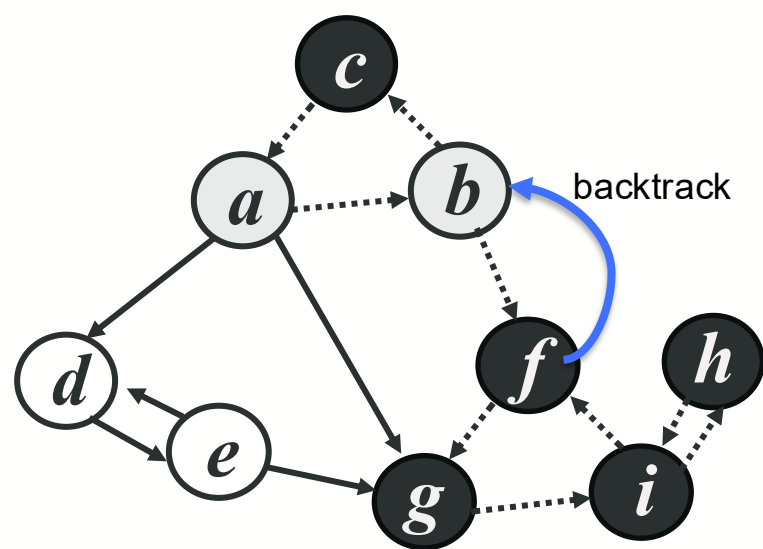
<i>v</i>	$\pi[v]$	<i>d</i> [<i>v</i>]	<i>f</i> [<i>v</i>]
<i>a</i>	<i>NIL</i>	1	
<i>b</i>	<i>a</i>	2	
<i>c</i>	<i>b</i>	3	4
<i>d</i>			
<i>e</i>			
<i>f</i>	<i>b</i>	5	
<i>g</i>	<i>f</i>	6	
<i>h</i>	<i>i</i>	8	9
<i>i</i>	<i>g</i>	7	10

Strong Components & DFS



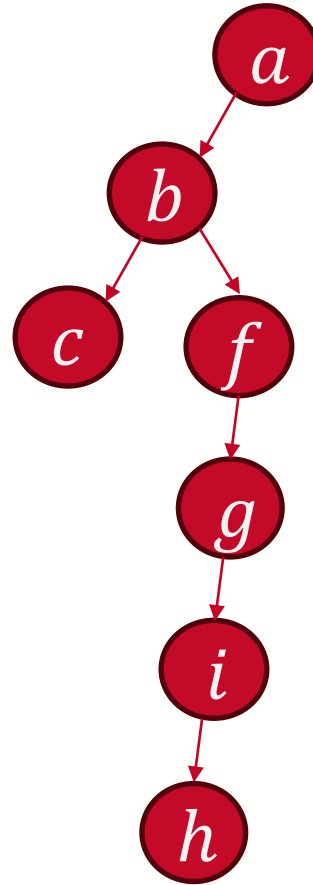
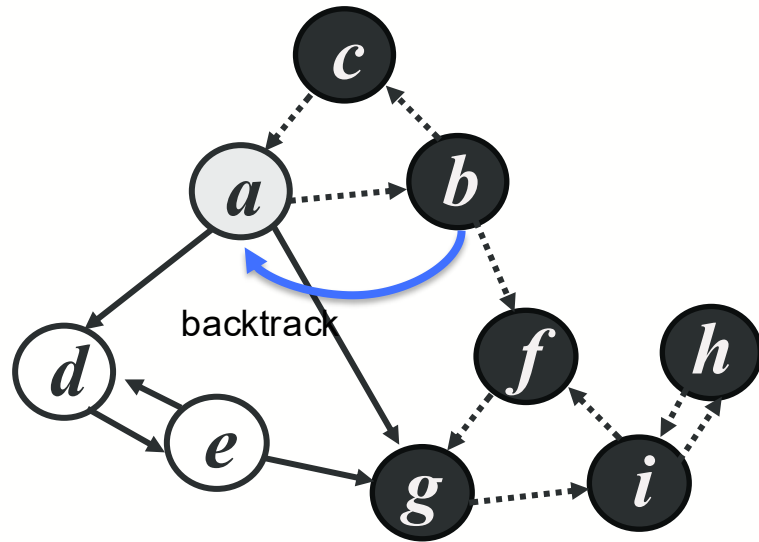
v	$\pi[v]$	$d[v]$	$f[v]$
<i>a</i>	<i>NIL</i>	1	
<i>b</i>	<i>a</i>	2	
<i>c</i>	<i>b</i>	3	4
<i>d</i>			
<i>e</i>			
<i>f</i>	<i>b</i>	5	
<i>g</i>	<i>f</i>	6	11
<i>h</i>	<i>i</i>	8	9
<i>i</i>	<i>g</i>	7	10

Strong Components & DFS



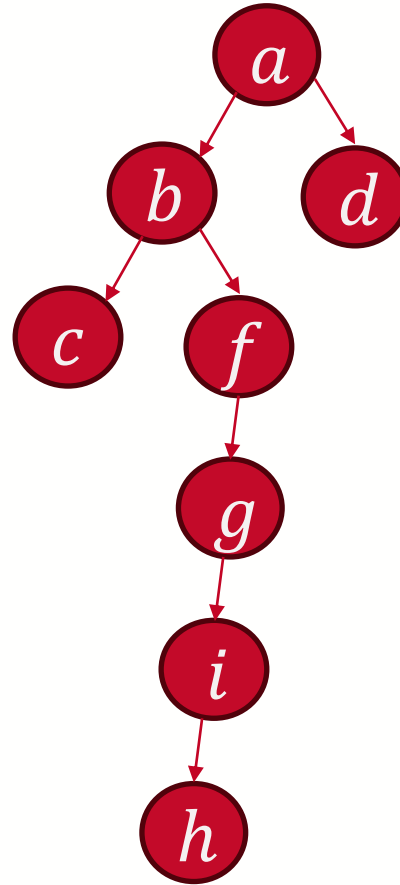
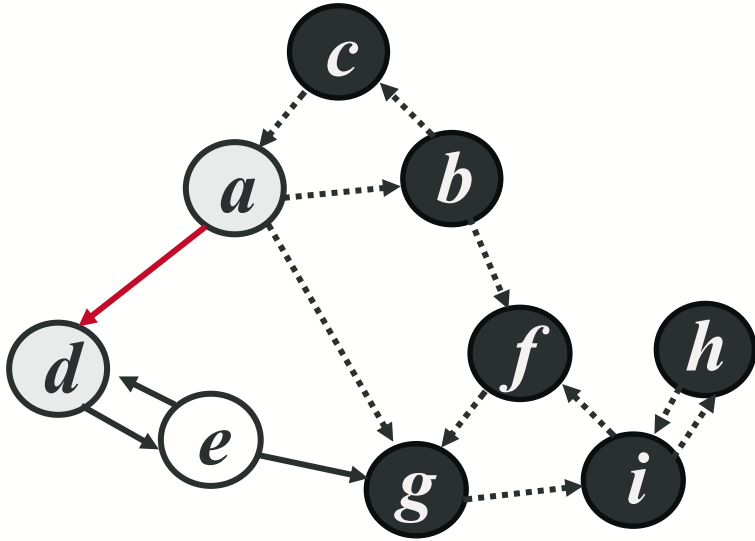
<i>v</i>	$\pi[v]$	<i>d</i> [<i>v</i>]	<i>f</i> [<i>v</i>]
<i>a</i>	<i>NIL</i>	1	
<i>b</i>	<i>a</i>	2	
<i>c</i>	<i>b</i>	3	4
<i>d</i>			
<i>e</i>			
<i>f</i>	<i>b</i>	5	12
<i>g</i>	<i>f</i>	6	11
<i>h</i>	<i>i</i>	8	9
<i>i</i>	<i>g</i>	7	10

Strong Components & DFS



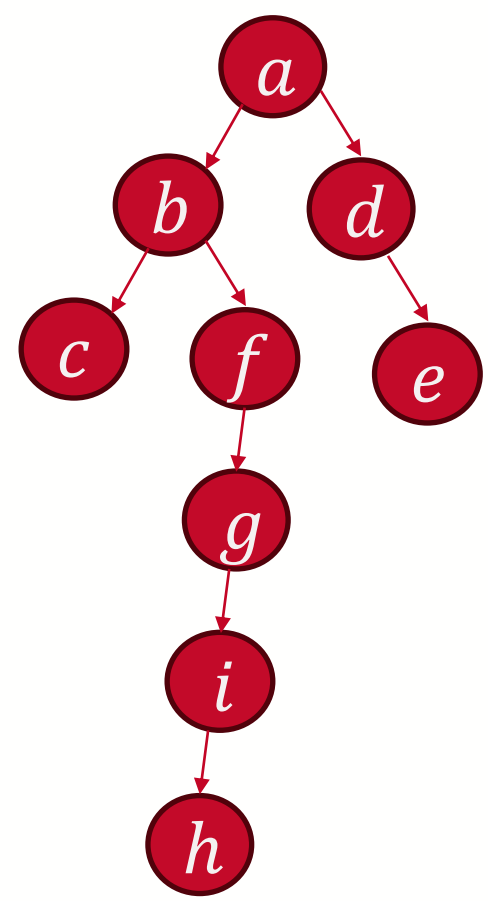
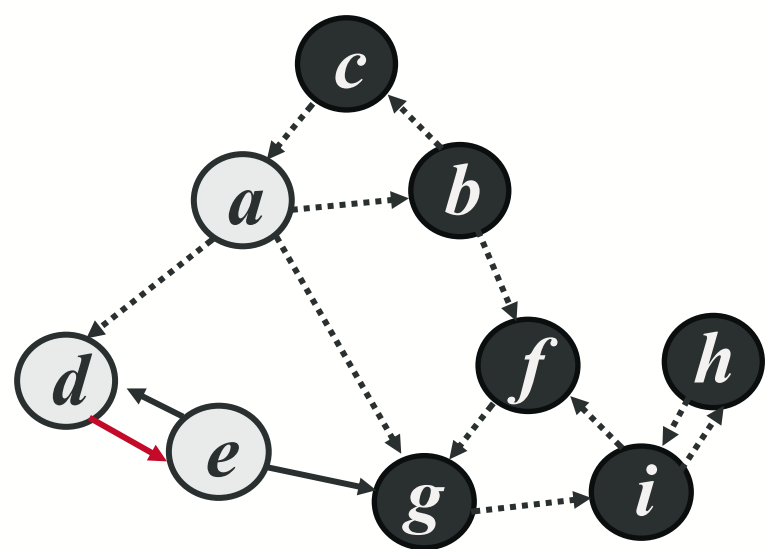
v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	
b	a	2	13
c	b	3	4
d			
e			
f	b	5	12
g	f	6	11
h	i	8	9
i	g	7	10

Strong Components & DFS



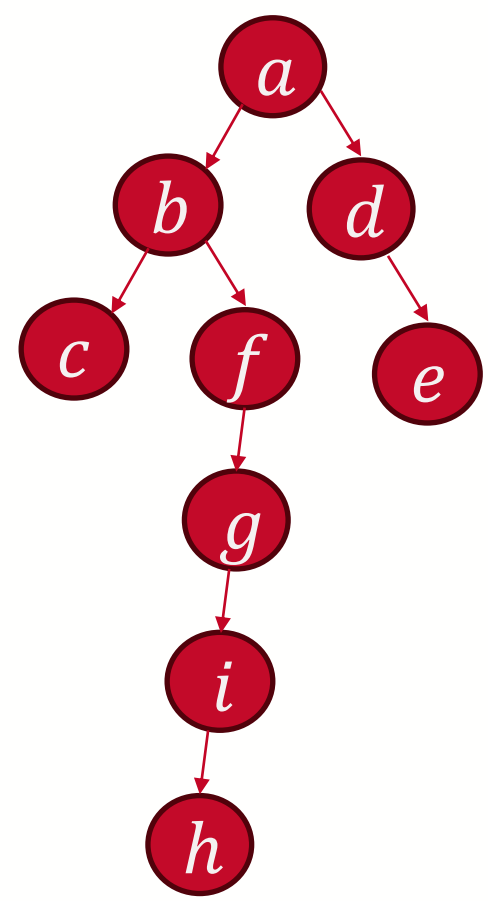
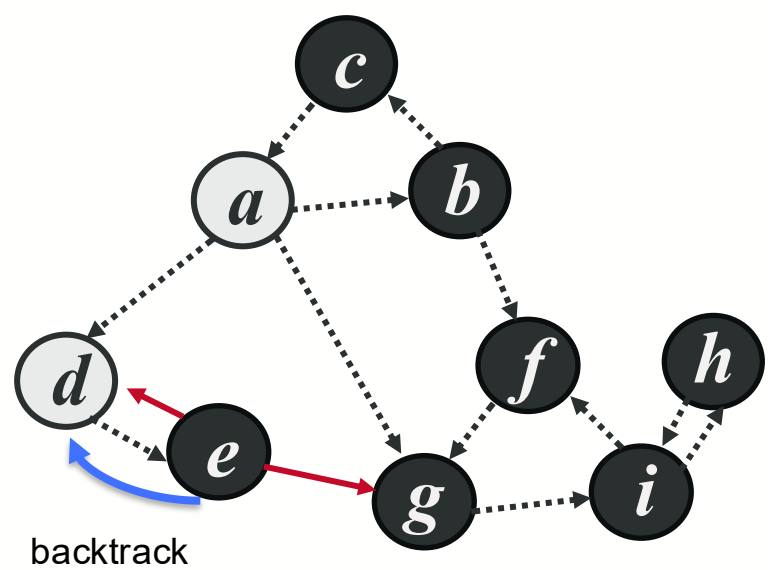
v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	
b	a	2	13
c	b	3	4
d	a	14	
e			
f	b	5	12
g	f	6	11
h	i	8	9
i	g	7	10

Strong Components & DFS



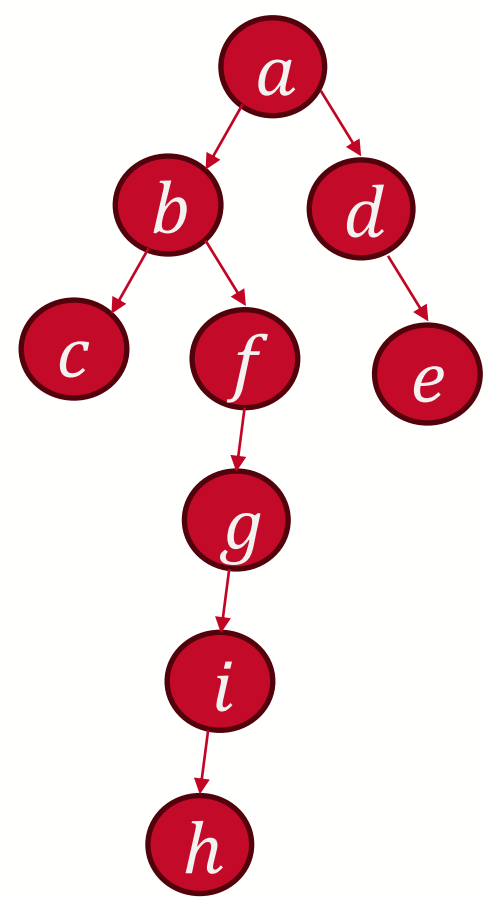
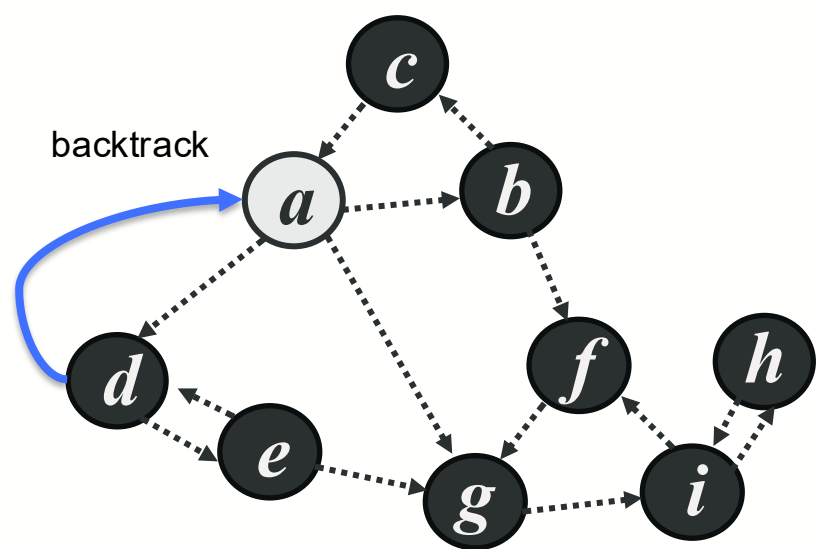
<i>v</i>	$\pi[v]$	<i>d</i> [<i>v</i>]	<i>f</i> [<i>v</i>]
<i>a</i>	<i>NIL</i>	1	
<i>b</i>	<i>a</i>	2	13
<i>c</i>	<i>b</i>	3	4
<i>d</i>	<i>a</i>	14	
<i>e</i>	<i>d</i>	15	
<i>f</i>	<i>b</i>	5	12
<i>g</i>	<i>f</i>	6	11
<i>h</i>	<i>i</i>	8	9
<i>i</i>	<i>g</i>	7	10

Strong Components & DFS



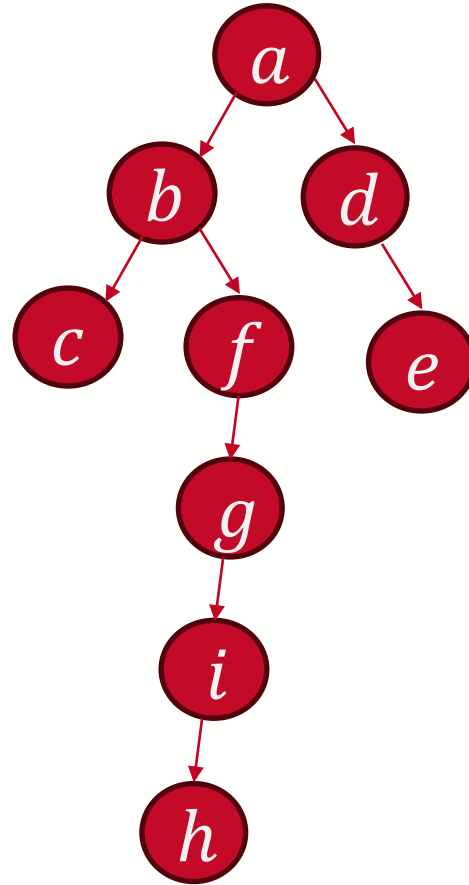
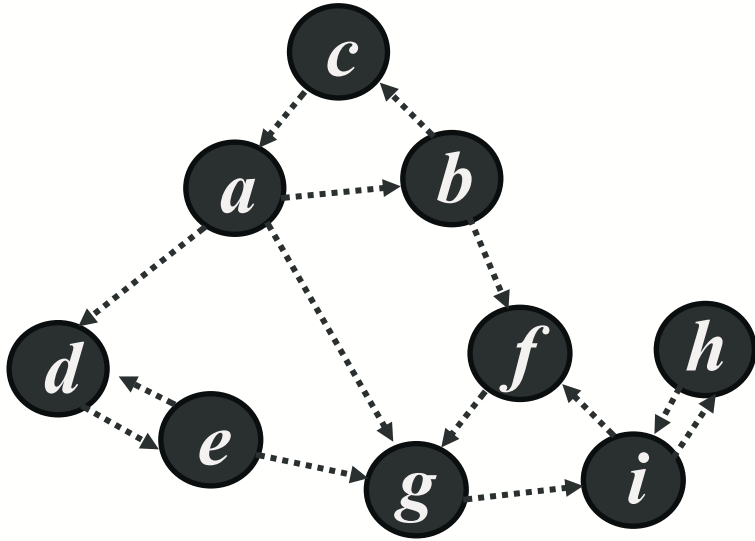
<i>v</i>	$\pi[v]$	<i>d</i> [<i>v</i>]	<i>f</i> [<i>v</i>]
<i>a</i>	<i>NIL</i>	1	
<i>b</i>	<i>a</i>	2	13
<i>c</i>	<i>b</i>	3	4
<i>d</i>	<i>a</i>	14	
<i>e</i>	<i>d</i>	15	16
<i>f</i>	<i>b</i>	5	12
<i>g</i>	<i>f</i>	6	11
<i>h</i>	<i>i</i>	8	9
<i>i</i>	<i>g</i>	7	10

Strong Components & DFS



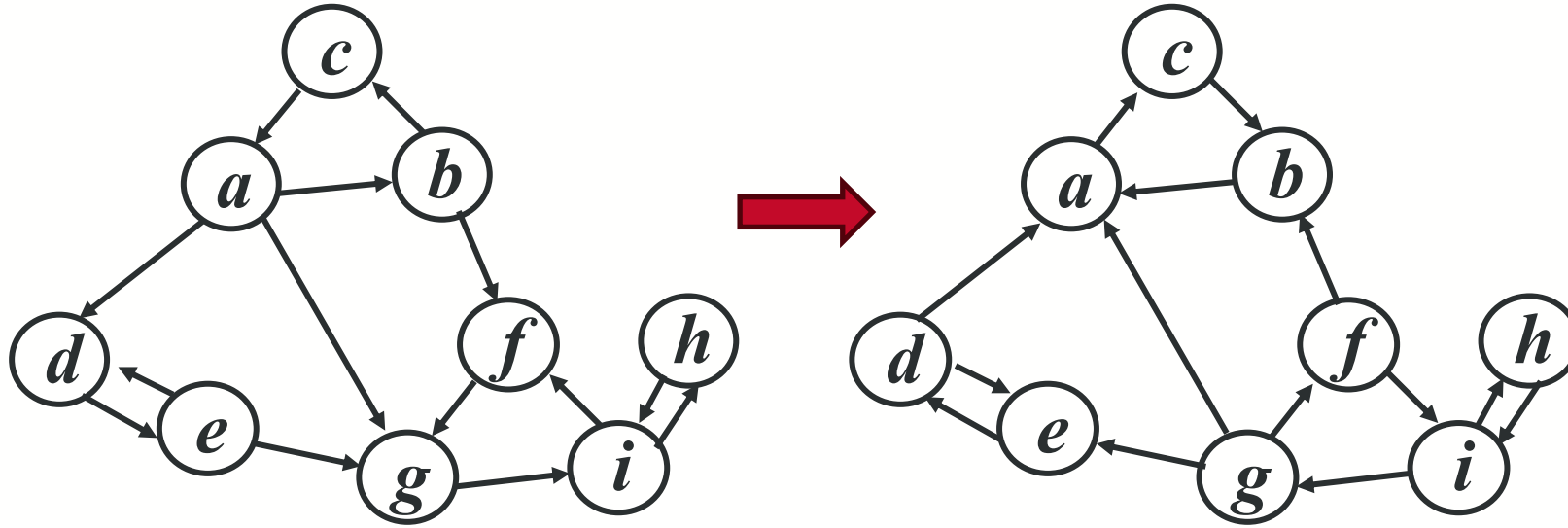
<i>v</i>	$\pi[v]$	<i>d</i> [<i>v</i>]	<i>f</i> [<i>v</i>]
<i>a</i>	<i>NIL</i>	1	
<i>b</i>	<i>a</i>	2	13
<i>c</i>	<i>b</i>	3	4
<i>d</i>	<i>a</i>	14	17
<i>e</i>	<i>d</i>	15	16
<i>f</i>	<i>b</i>	5	12
<i>g</i>	<i>f</i>	6	11
<i>h</i>	<i>i</i>	8	9
<i>i</i>	<i>g</i>	7	10

Strong Components & DFS



v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	18
b	a	2	13
c	b	3	4
d	a	14	17
e	d	15	16
f	b	5	12
g	f	6	11
h	i	8	9
i	g	7	10

R = Reverse(G)

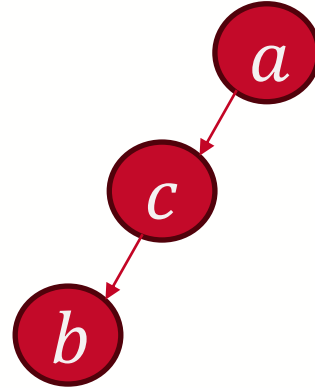
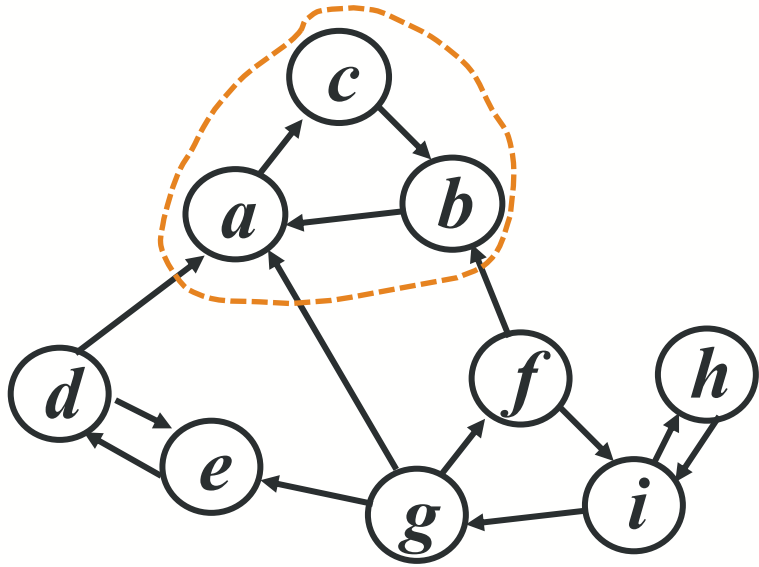


Decreasing order of $f[u]$

$a \rightarrow d \rightarrow e \rightarrow b \rightarrow f \rightarrow g \rightarrow i \rightarrow h \rightarrow c$

v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	18
b	a	2	13
c	b	3	4
d	a	14	17
e	d	15	16
f	b	5	12
g	f	6	11
h	i	8	9
i	g	7	10

DFS(R)

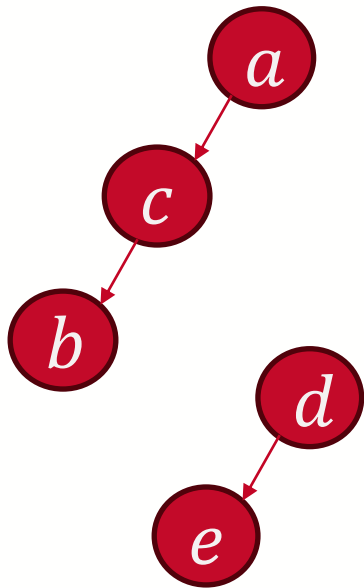
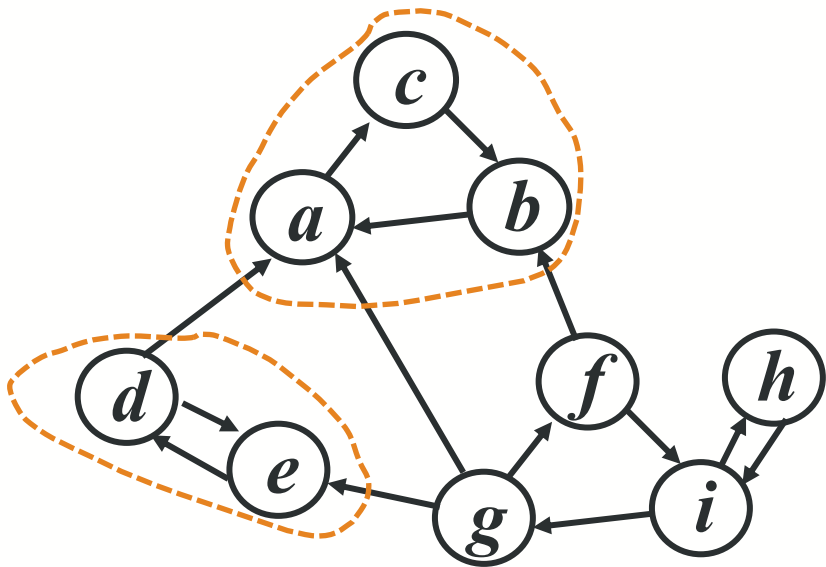


Decreasing order of $f[u]$

$a \rightarrow d \rightarrow e \rightarrow b \rightarrow f \rightarrow g \rightarrow i \rightarrow h \rightarrow c$

v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	6
b	c	3	4
c	a	2	5
d			
e			
f			
g			
h			
i			

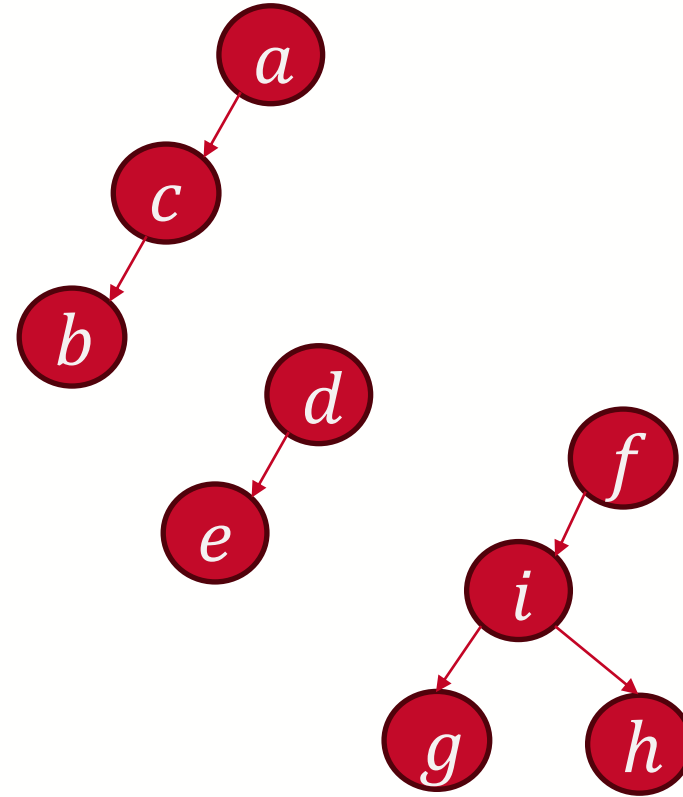
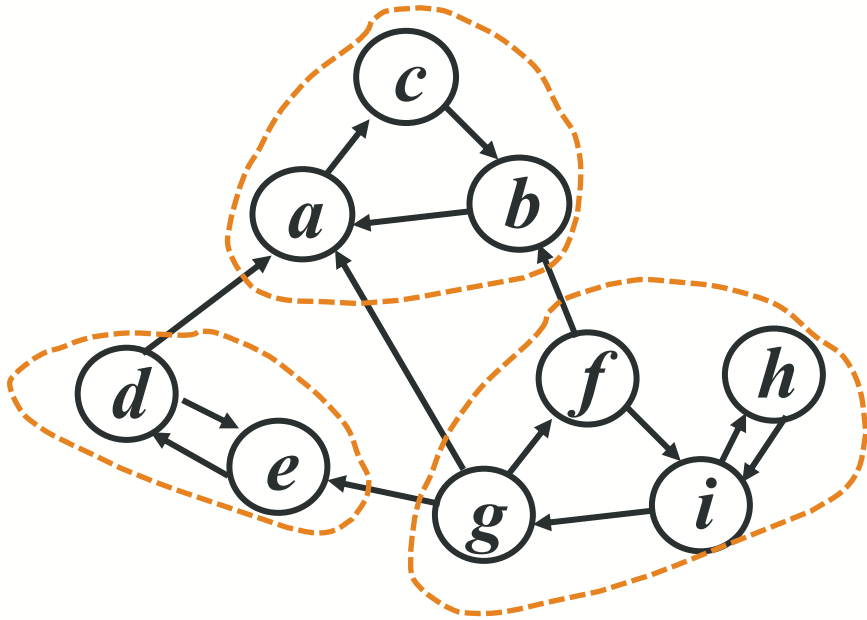
DFS(R)



Decreasing order of $f[u]$
 $a \rightarrow \textcolor{red}{d} \rightarrow e \rightarrow b \rightarrow f \rightarrow g \rightarrow i \rightarrow h \rightarrow c$

v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	6
b	c	3	4
c	a	2	5
d	NIL	7	10
e	d	8	9
f			
g			
h			
i			

DFS(R)



Decreasing order of $f[u]$

$a \rightarrow d \rightarrow e \rightarrow b \rightarrow \mathbf{f} \rightarrow g \rightarrow i \rightarrow h \rightarrow c$

v	$\pi[v]$	$d[v]$	$f[v]$
a	NIL	1	6
b	c	3	4
c	a	2	5
d	NIL	7	10
e	d	8	9
f	NIL	11	18
g	i	13	14
h	i	15	16
i	f	12	17

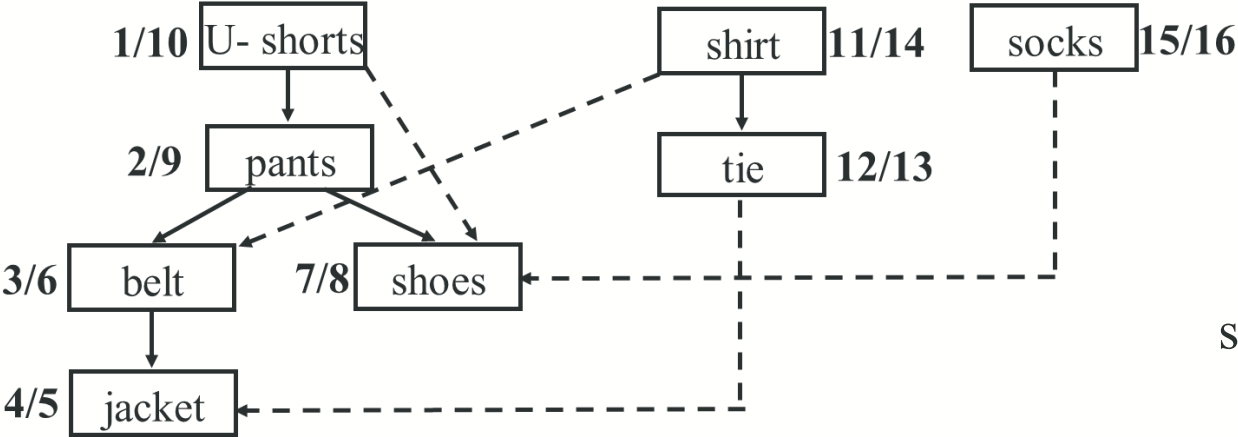
Strongly Connected Components for DAGs

- Do DAGs have strongly connected components?

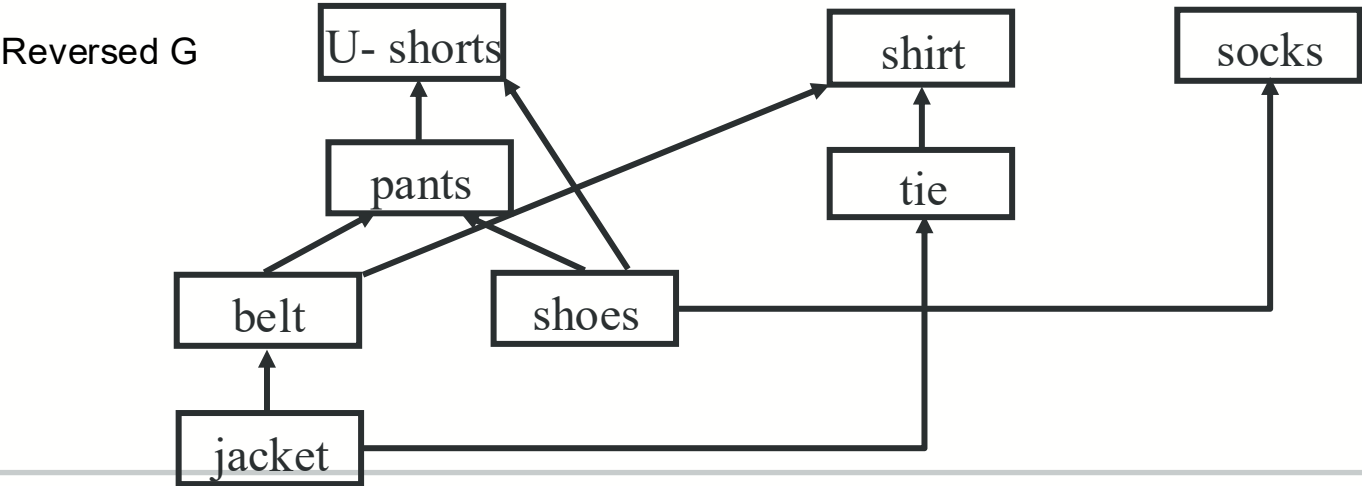


- DAG Definition: No cycles, directed edges
- SCC Definition: Maximal subgraph where every node is mutually reachable

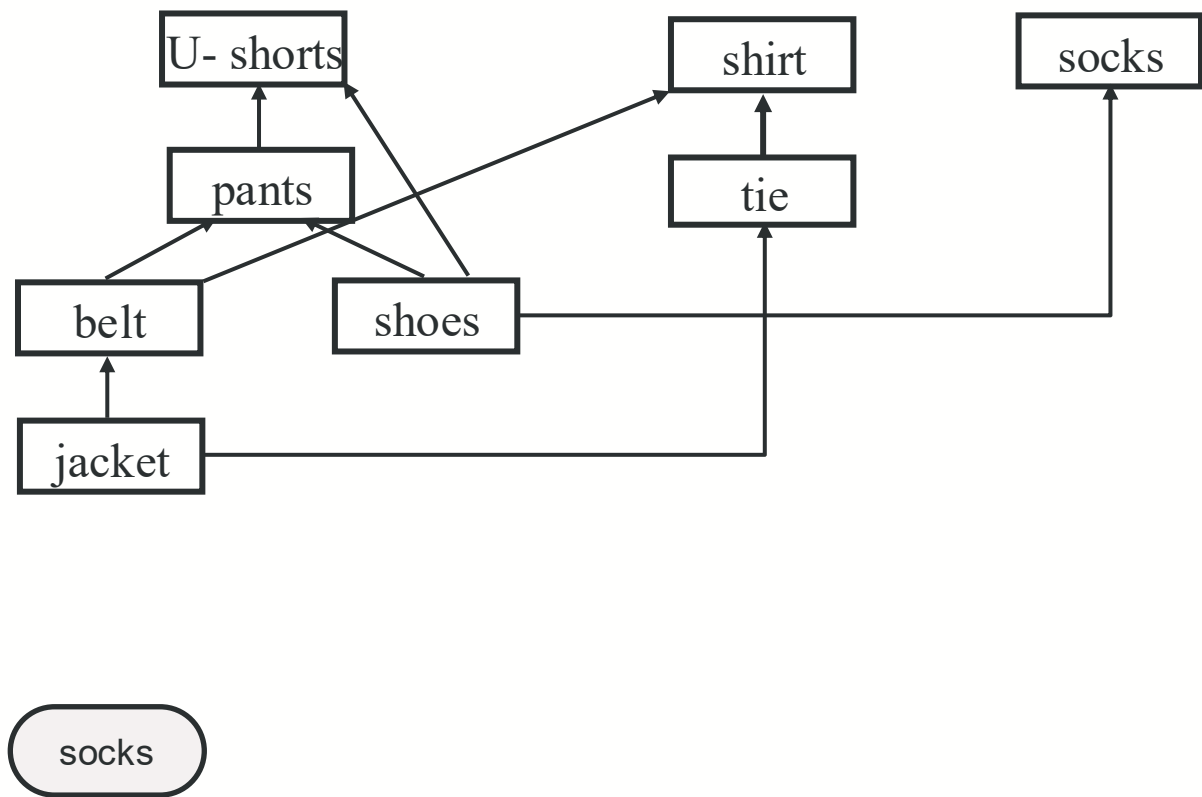
Another Example – Acyclic Graph



Final Ordering (decreasing order of $f[u]$):
socks, shirt, tie, u-shorts, pants, shoes, belt, jacket



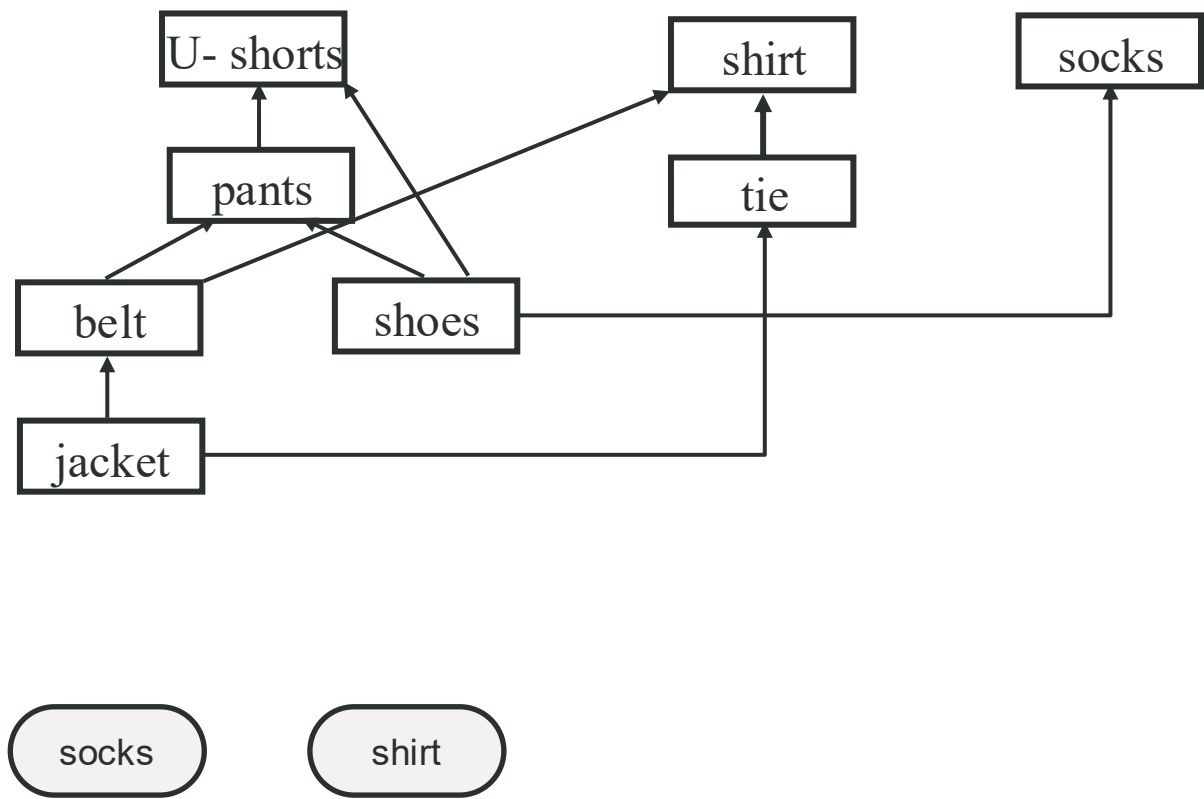
Another Example – Acyclic Graph



Final Ordering (decreasing order of $f[u]$):
socks, shirt, tie, u-shorts, pants, shoes, belt, jacket

vertex	d	f
U-shorts		
Pants		
Belt		
Shoes		
Jacket		
Shirt		
Tie		
Socks	1	2

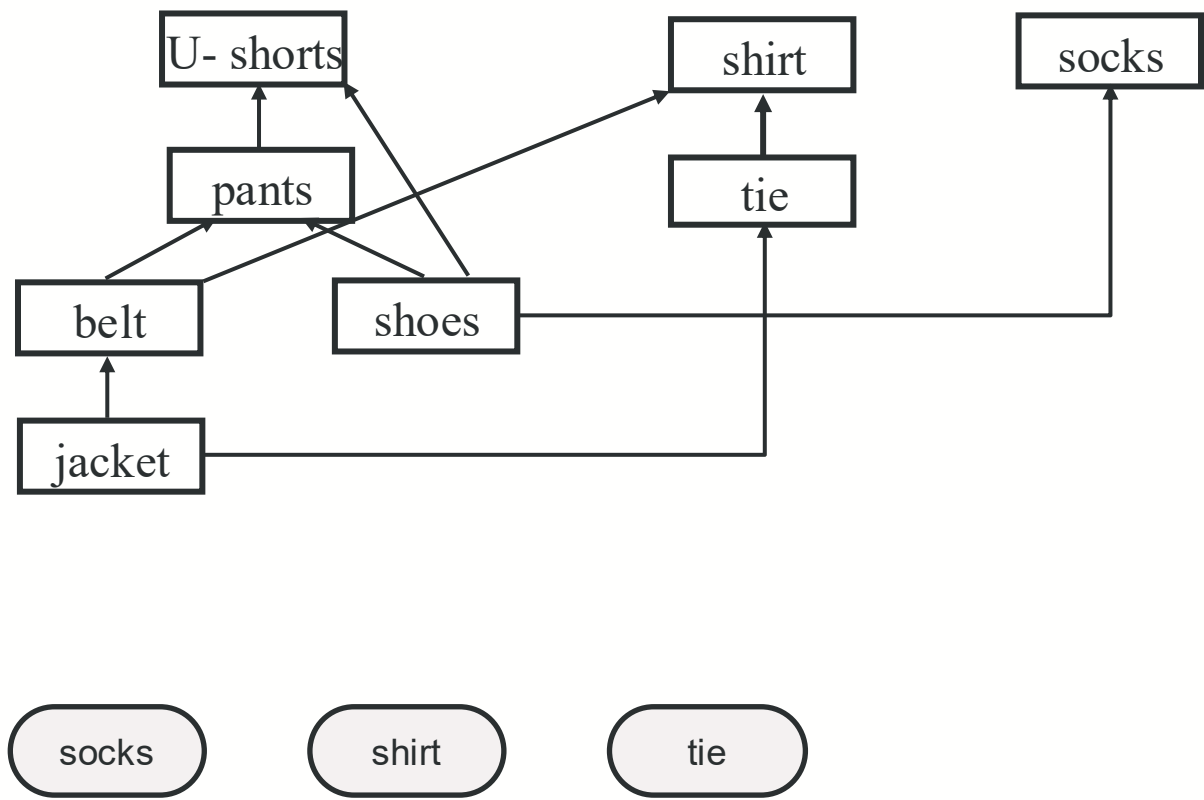
Another Example – Acyclic Graph



Final Ordering (decreasing order of $f[u]$):
socks, shirt, tie, u-shorts, pants, shoes, belt, jacket

vertex	d	f
<i>U-shorts</i>		
<i>Pants</i>		
<i>Belt</i>		
<i>Shoes</i>		
<i>Jacket</i>		
<i>Shirt</i>	3	4
<i>Tie</i>		
<i>Socks</i>	1	2

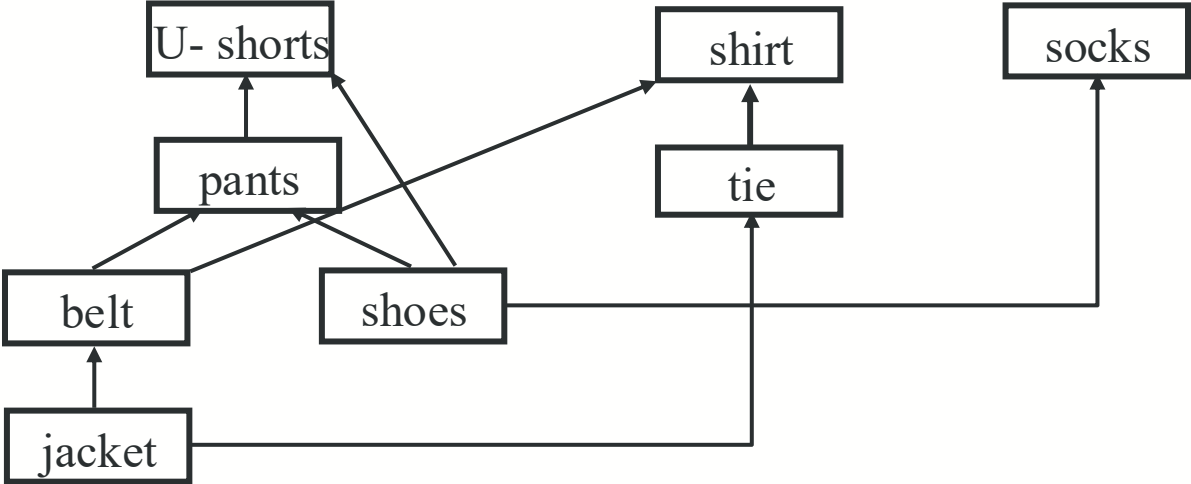
Another Example – Acyclic Graph



Final Ordering (decreasing order of $f[u]$):
socks, shirt, tie, u-shorts, pants, shoes, belt, jacket

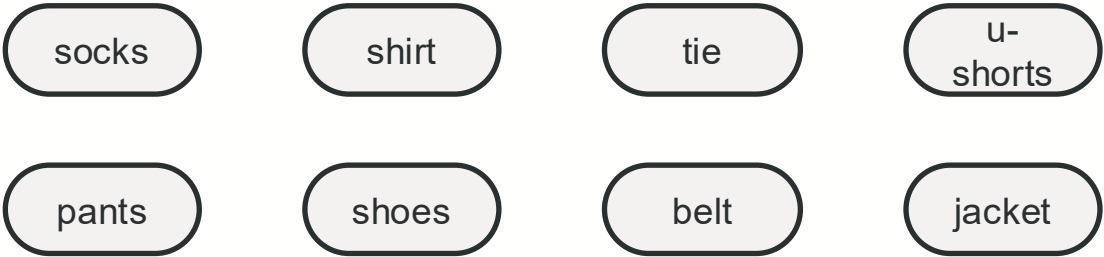
vertex	d	f
U-shorts		
Pants		
Belt		
Shoes		
Jacket		
Shirt	3	4
Tie	5	6
Socks	1	2

Another Example – Acyclic Graph



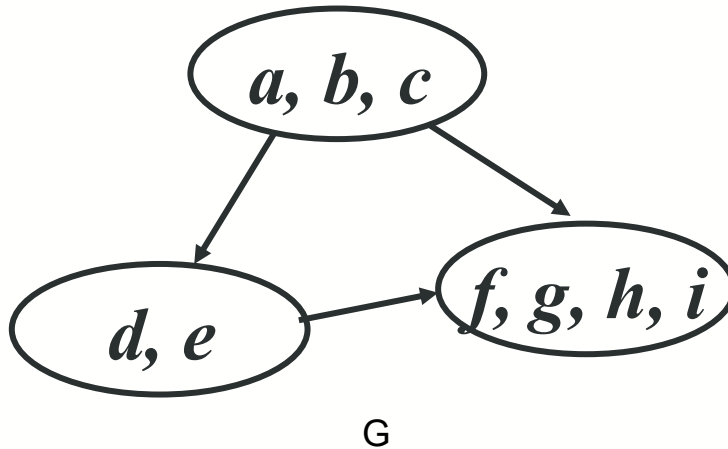
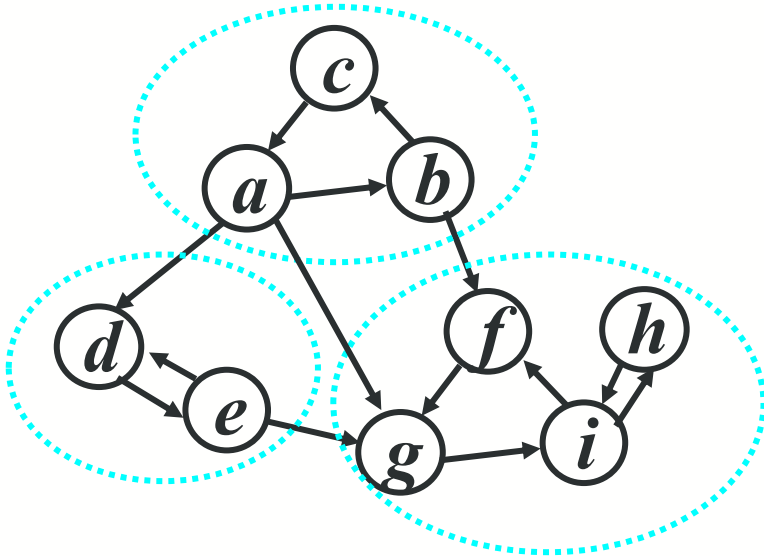
Final Ordering (decreasing order of $f[u]$):
socks, shirt, tie, u-shorts, pants, shoes, belt, jacket

vertex	d	f
U-shorts	7	8
Pants	9	10
Belt	13	14
Shoes	11	12
Jacket	15	16
Shirt	3	4
Tie	5	6
Socks	1	2



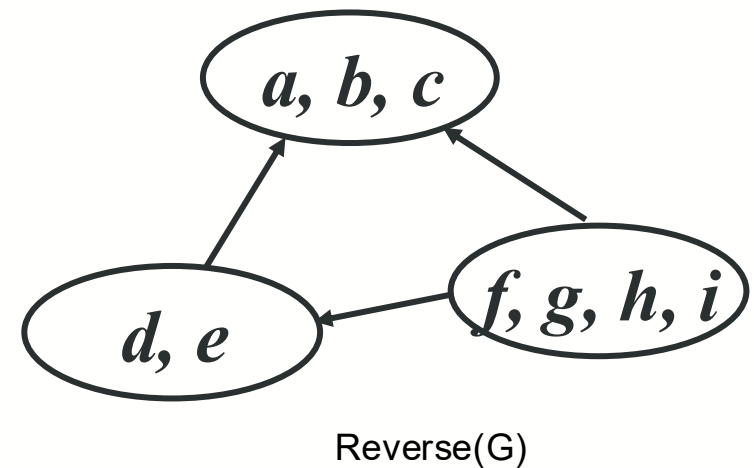
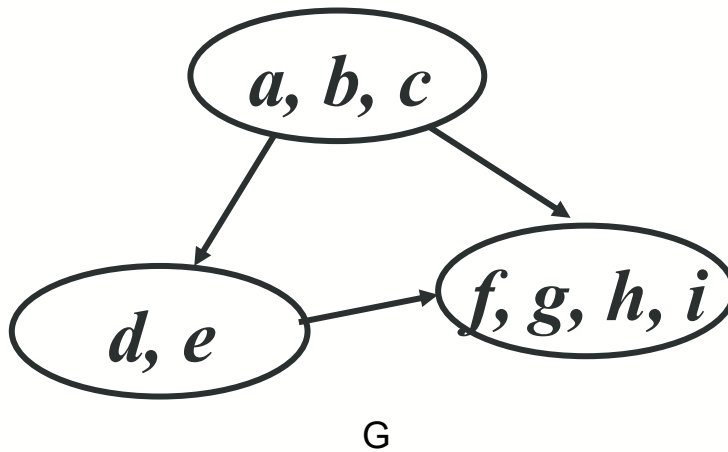
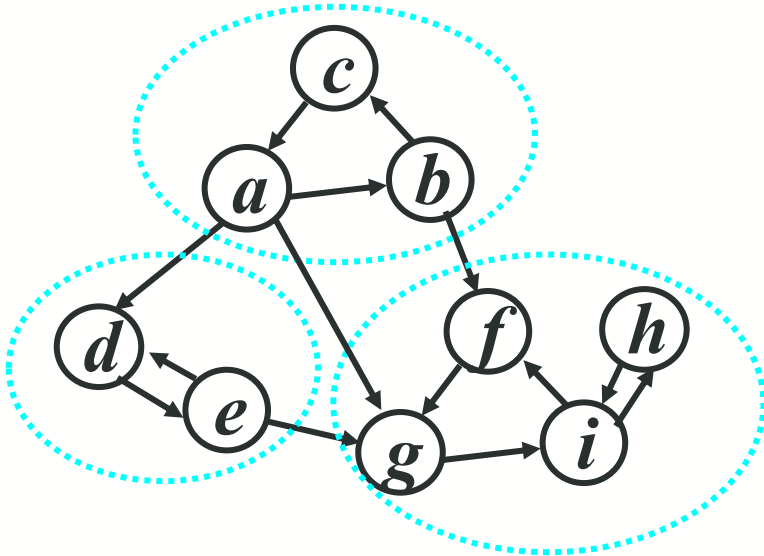
Correctness (Intuition)

- Topological sort only works on DAGs
- It orders nodes so that for every directed edge $u \rightarrow v$, node u comes before v in the ordering
- In the original graph: The components with all incoming edges are at the end of the topological sort
 - $(a,b,c) \rightarrow (d,e) \rightarrow (f,g,h,i)$



Correctness (Intuition)

- At least one component must have all edges incoming. Want to visit that one first, to avoid leakage
- If you reversed the edges, that component would be the first one in a topological sort
 - $(f,g,h,i) \rightarrow (d,e) \rightarrow (a,b,c)$



Correctness (Intuition)

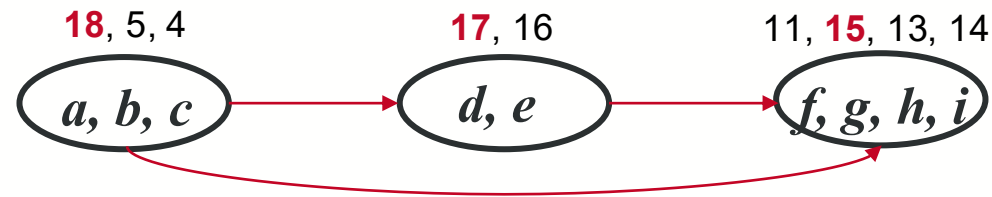
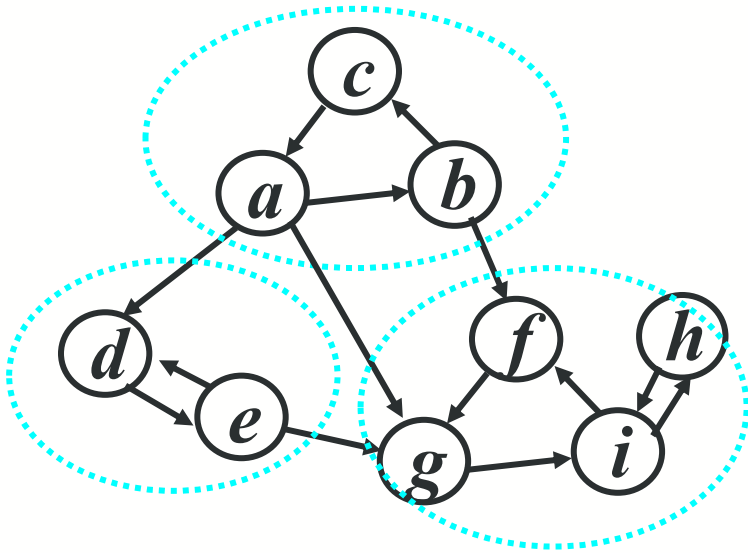
- Fortunately, reversing the edges doesn't change the strongly connected components. That is, mutual reachability between two vertices is the same for G and G^T .
- So: Do a topological sort, then reverse the edges, then do DFS in decreasing order of finishing time (i.e., reverse sorted $f[u]$)

Correctness

- Refer to the textbook for a formal proof. Here is some intuition why it works:
 - Strong components are not affected by reversing all edges in the digraph. i.e. mutual reachability between two vertices are the same in G and G^T
 - The main idea is to visit the strong components in a reverse topological order so to avoid “leak”.
 - If we simply visit the components in reverse topological order, we would visit the vertices in increasing order of finish time, starting with the lowest finishing time. (This won't work)
 - Consider max finish time in each component. Visit the components related to the topological order.

Lemma

- Consider a digraph G on which DFS has been run. Label each component with the maximum finish time of all the vertices in the component, and sort these in decreasing order. Then this order is a topological order for the component digraph.



vertex	f
<i>a</i>	18
<i>b</i>	5
<i>c</i>	4
<i>d</i>	17
<i>e</i>	16
<i>f</i>	11
<i>g</i>	15
<i>h</i>	13
<i>i</i>	14

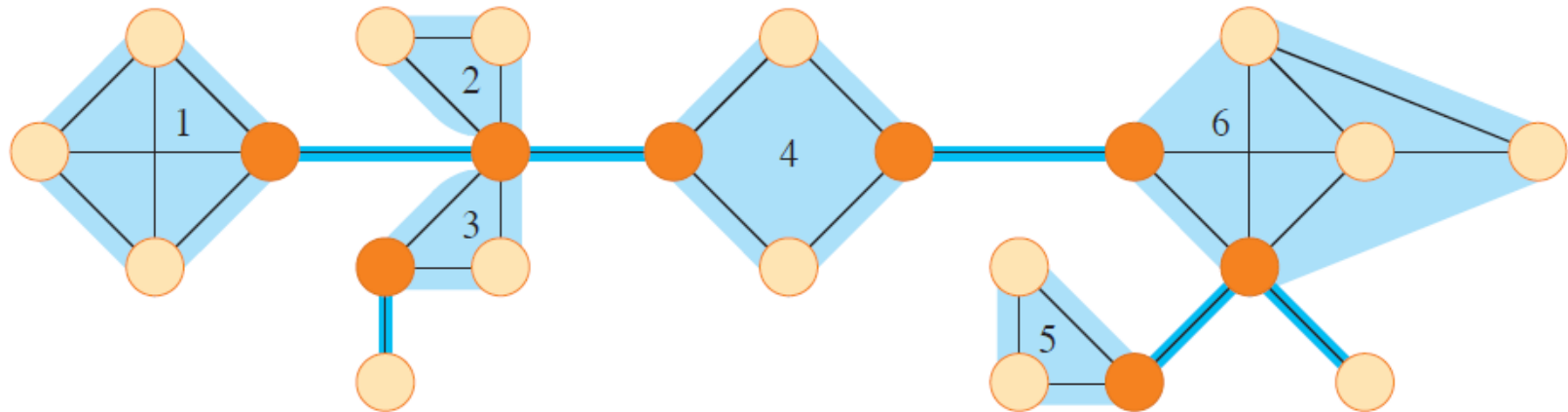
Articulation Points

(In Undirected Graphs)

Definitions

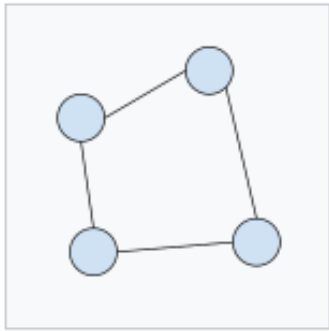
- *Articulation Point* (or *cut vertex*): any vertex whose removal (together with any incident edges) results in a disconnected graph
- *Bridge*: an edge whose removal results in a disconnected graph
- **Importance**: crucial for understanding the vulnerability and connectivity of a network.

Orange vertices: articulation points
Dark blue edges: bridges

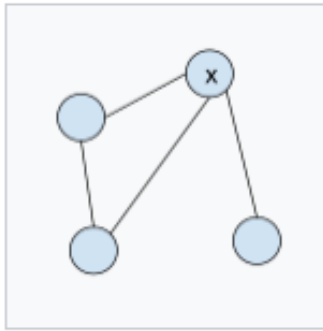


Definitions

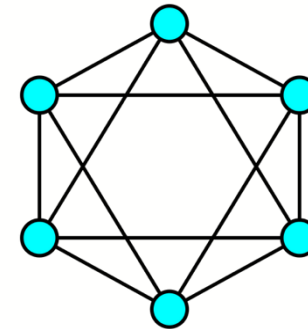
- *Biconnected*: containing no articulation points. In other words, the graph remains connected regardless of the removal of any single vertex.
 - Biconnected graphs are more robust to vertex removal, making them more resilient in network structures
- In general a graph is k -connected, if k vertices must be removed to disconnect the graph.



A biconnected graph on four vertices and four edges



A graph that is not biconnected. The removal of vertex x would disconnect the graph.

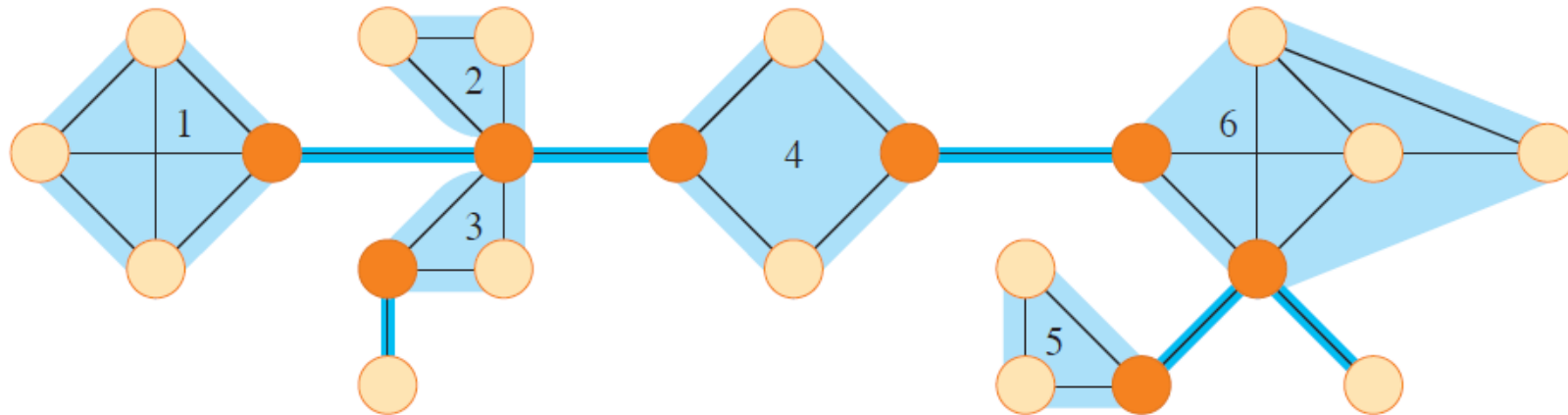


Connectivity: 4

Image: wikipedia

Definitions

- *Biconnected Components*: a maximal set of edges s.t. any 2 edges in the set lie on a common simple cycle
- Biconnected components help identify parts of the graph that are internally connected and can be separated from the rest by removing a single articulation point.



Biconnected components: edges in the light blue regions

Articulation Points and DFS

- *Basic idea:* use the structures in the DFS tree to find articulation points.
- Assume G is connected, so there is only one tree in the DFS forest. Since G is undirected, we don't distinguish between forward & back edges -- we call them both back edges. There are no cross edges.

Observations

- 1 An internal vertex u of the DFS tree (other than the root) is an articulation point if and only if there exists a subtree rooted at a child of u such that there is no back edge from any vertex in this subtree to a proper ancestor of u .
- 2 A leaf of the DFS tree is never an articulation point.
- 3 The root of the DFS tree is an articulation point if and only if it has two or more children.
 - Removing the root disconnects the subtrees from each other, resulting in multiple disconnected components.

Articulation Points by DFS

- Observation 2 & 3 are easy to check.
- Observation 1: is there some back edge from some subtree to an ancestor of a given vertex? Keep track of back edge that goes highest in the tree (closest to the root). We do this by keeping track of “discovery time”: i.e. find the back edge (v, w) with the smallest $d[w]$.

Basic Algorithm

- Process nodes in postorder during DFS
- Update $low[v]$ values dynamically based on back edges and subtree connections.
- For each node v visited, calculate $low[v]$ defined as

$$low[v] = \min(d[v], d[w])$$

(u, w) : a back edge for some descendant u of v to an ancestor w

$low[v]$ represents the earliest reachable ancestor from the subtree of v , using back edges.

- Articulation points are determined as follows:
 - Root of $T(G)$, if and only if it has more than 1 child
 - A node v other than the root, if and only if has a child x such that $low[x] \geq d[v]$

ArtPt(u)

```
1. Color[ $u$ ] = gray
2. Low[ $u$ ] = d[ $u$ ] = ++time
3. for each ( $v \in \text{Adj}(u)$ ) {
4.     if (color[ $v$ ] == white) {                                //  $v$  is not visited yet. ( $u, v$ ) is a tree edge
5.         pred[ $v$ ] =  $u$ 
6.         ArtPt( $v$ )
7.         Low[ $u$ ] = min(Low[ $u$ ], Low[ $v$ ])                      // update Low[ $u$ ]
8.         if (pred[ $v$ ] == NULL) {                               // root: apply Observation 3
9.             if ( $v$  is  $u$ 's second++ child)
10.                Add  $u$  to set of articulation points
11.        }
12.        else if (Low[ $v$ ]  $\geq$  d[ $u$ ]) {                          // internal node: apply Obs. 1
13.            Add  $u$  to set of articulation points
14.        }
15.    } else if ( $v \neq \text{pred}[u]$ )                             //  $v$  is not white (already visited) and  $v$  is not the predecessor of  $u$ : ( $u, v$ ) is a back edge
16.        Low[ $u$ ] = min(Low[ $u$ ], d[ $v$ ])                        // update Low[ $u$ ]
17. }
```


DFS Tree

$\text{low}[b] = \min(\text{low}[b], \text{low}[a])$
 $= \min(1, 1) = 1$

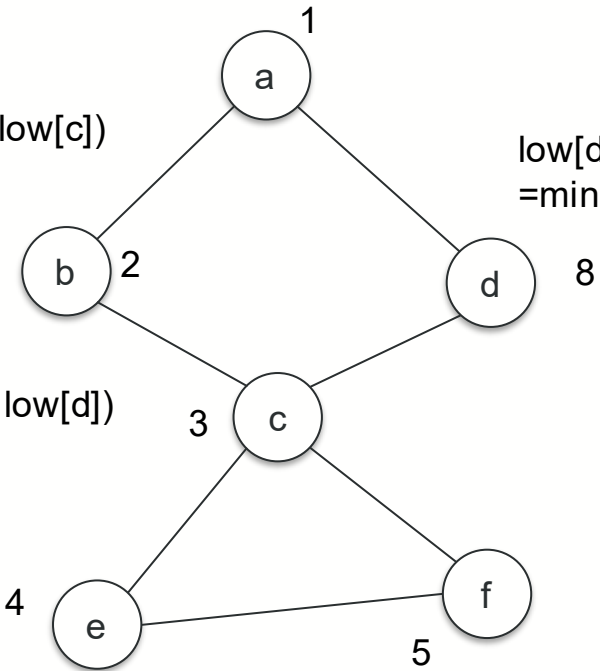
$\text{low}[b] = \min(\text{low}[b], \text{low}[c])$
 $= \min(2, 1) = 1$

$\text{low}[c] = \min(\text{low}[c], \text{low}[d])$
 $= \min(3, 1) = 1$

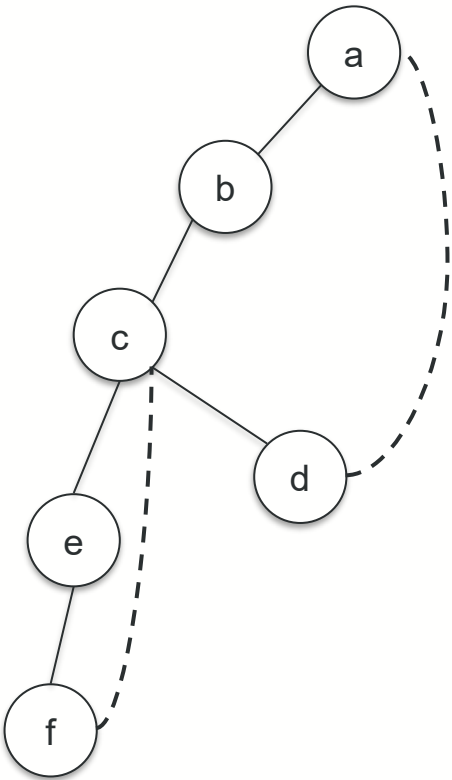
$\text{low}[e] = \min(\text{low}[e], \text{low}[f])$
 $= \min(4, 3) = 3$

$\text{low}[f] = \min(\text{low}[f], \text{d}[c]) = 3$

$\text{low}[d] = \min(\text{low}[d], \text{d}[a])$
 $= \min(8, 1) = 1$



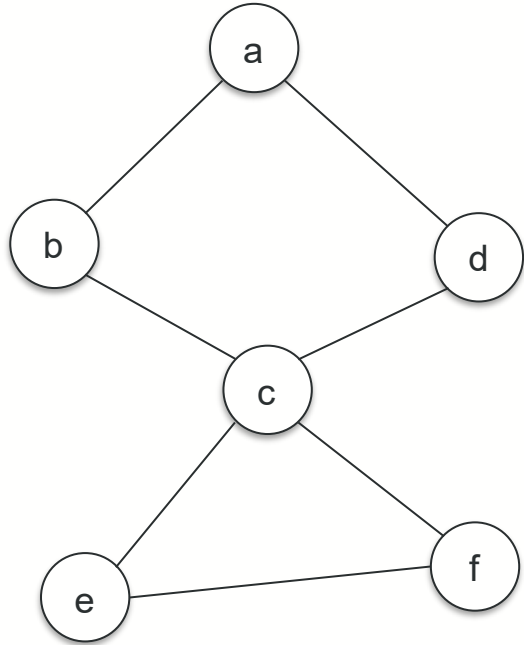
G



T(G)

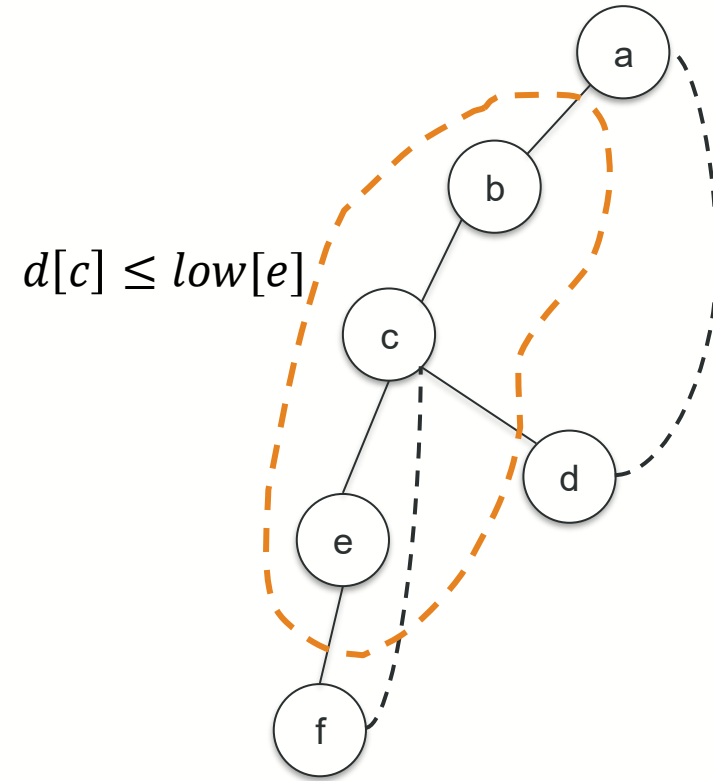
vertex	d	Low
a	1	1
b	2	1
c	3	1
d	6	1
e	4	3
f	5	3

DFS Tree Post-order Traversal



G

vertex	d	Low
a	1	1
b	2	1
c	3	1
d	6	1
e	4	3
f	5	3



In the DFS tree, for an internal node v and its child x of v
if $low[x] \geq d[v]$, then v is an articulation point